



Estrela Gestão

Documentação técnica completa — Sistema de
Estoque e Pedidos (Fase 1)

Estrela América do Sul · FastAPI + HTMX + PostgreSQL · 100% local
e offline

Consolidado em 24/06/2026 · 16 documentos

Conteúdo

01 Contexto-raiz e padrões de código

02 Índice e guia do projeto

03 Regras de negócio

04 Arquitetura e fundação

05 Modelo de dados e migrations

06 ETL — importação de planilhas

07 Módulo de estoque

08 Módulo de pedidos e financeiro

09 Frontend, UI e PWA

10 Infraestrutura, deploy e go-live

11 Dicionário de dados

12 Runbook do servidor

13 Checklist de go-live

14 Disaster recovery

15 LGPD — operador

16 Nobreak e NUT

CLAUDE.md — Estrela Gestão (Sistema de Estoque e Pedidos)

Para o agente Claude Code: este arquivo é o contexto-raiz do projeto. Leia-o inteiro antes de escrever qualquer código. Salve-o como `CLAUDE.md` na raiz do repositório. Sempre que houver dúvida de padrão, este documento vence. Para operações de Git, consulte a skill `git-workflow`. O documento de requisitos completo do cliente (`planejamento-sistema-estoque-pedidos.md`) é a referência funcional; aqui está o que você precisa para **construir**.

1. O que estamos construindo

Sistema **100% local e offline** de **controle de estoque e pedidos** para a empresa Estrela (atacado import/export), rodando em um servidor instalado no cliente e acessado por **até 10 terminais** pela rede interna, via navegador em **modo aplicativo** (PWA — ícone próprio, tela cheia, sem cara de site).

- **Não depende de internet** para operar. Internet só para manutenção remota e backup offsite.
- **Sem instalação nas estações** — só o servidor roda o sistema.
- Volume: **80–100 pedidos/dia**, estoque com **centenas de milhares de SKUs** e alto giro.
- **4 perfis de acesso:** Admin, Vendedor, Financeiro, Funcionário.
- Nasce preparado para a **Fase 2** (catálogo web + WhatsApp com IA), sem retrabalho de modelagem.

A "cara" do produto já foi prototipada e aprovada (dashboard com cards/KPIs, tabelas densas, paleta dourada da marca — **dourado primário** `#B98A19`, fundo creme `#F6F2E8`, sidebar marrom escuro `#211B0F`).

2. Stack definitiva

Camada	Tecnologia	Observação
Linguagem	Python 3.12	
Gerenciador de pacotes	uv (Astral)	Rápido; <code>uv add</code> , <code>uv run</code> . Alternativa: Poetry
Framework web/API	FastAPI	
Servidor ASGI	Uvicorn (dev) / Gunicorn + UvicornWorker (prod)	
Banco	PostgreSQL 16 (container)	extensão <code>pg_trgm</code> para busca
ORM	SQLAlchemy 2.0 (sync) + psycopg 3	Sync , não async — ver §4
Migrations	Alembic	
Validação/Schemas	Pydantic v2 + pydantic-settings	config via <code>.env</code>
Auth	PyJWT + passlib[argon2]	RBAC via dependencies do FastAPI

Camada	Tecnologia	Observação
Templates (UI)	Jinja2 + HTMX + Alpine.js	server-rendered; HTMX p/ interatividade
CSS	Tailwind CSS (Standalone CLI, sem Node)	build do CSS sem depender de JS toolchain
PWA	<code>manifest.webmanifest</code> + service worker (network-first)	habilita o "modo programa" nos terminais
Agendamento	APScheduler	jobs internos (ex.: checagens); backup via pgBackRest fora do app
Testes	pytest + httpx (TestClient) + pytest-cov	
Lint + Format	Ruff	lint e format numa ferramenta só
Container	Docker + Docker Compose	app + Postgres
Proxy	Caddy	HTTPS interno automático (<code>https://sistema.local</code>)
Acesso remoto (manutenção)	Tailscale	sem expor portas à internet

Por que NÃO usar async: o volume é baixo (10 usuários) e o sistema é CRUD-pesado com transações críticas (reserva e baixa de estoque). SQLAlchemy **sync** dá transações previsíveis, menos pegadinhas e é mais do que suficiente em performance. FastAPI executa endpoints `def` (sync) em threadpool automaticamente.

Por que NÃO Supabase / ORM mágico: PostgreSQL puro + SQLAlchemy cobre 100% do caso com controle total e operação simples numa máquina que precisa rodar sozinha por meses.

Por que Jinja2 + HTMX em vez de SPA: stack única em Python, sem build de JS, server-rendered (latência mínima na rede local), e HTMX entrega busca instantânea e atualização parcial de tabelas sem a complexidade de um frontend separado. A camada de **services** já fica pronta para expor **JSON** na Fase 2 (catálogo/WhatsApp).

3. Estrutura do projeto

```

estrela-gestao/
├── CLAUDE.md                # este arquivo
├── pyproject.toml           # deps (uv)
├── .env.example
├── docker-compose.yml       # app + postgres (+ caddy em prod)
├── Dockerfile
├── alembic.ini
├── alembic/
│   └── versions/            # migrations
├── app/
│   ├── main.py              # cria o FastAPI, monta routers/web, static, middlewares
│   ├── core/
│   │   ├── config.py        # Settings (pydantic-settings)
│   │   ├── database.py      # engine + SessionLocal + Base
│   │   └── security.py      # hash de senha (argon2), criar/validar JWT
│   └── deps/
│       ├── db.py            # get_db() -> Session
│       └── auth.py          # get_current_user(), require_role(...)

```

```

├── models/                # SQLAlchemy ORM (1 arquivo por agregado)
├── schemas/               # Pydantic (request/response/DTOs)
├── repositories/         # acesso a dados (queries SQLAlchemy)
├── services/             # regras de negócio (lógica central)
├── controllers/          # handlers: validam, chamam service, montam resposta
├── routers/              # endpoints JSON (Fase 2 e integrações)
├── web/
│   ├── routes/           # rotas que devolvem HTML (Jinja2 + HTMX)
│   └── templates/        # .html (base, páginas, fragmentos htmx)
├── static/
│   ├── css/              # output do Tailwind
│   ├── js/               # htmx.min.js, alpine.min.js
│   ├── icons/            # ícones do PWA (Estrela Gestão)
│   ├── manifest.webmanifest
│   └── sw.js              # service worker (network-first)
├── importer/             # ETL das planilhas (staging, validação, carga)
├── tests/
│   ├── conftest.py       # fixtures: db de teste, client, usuários por perfil
│   ├── test_estoque.py
│   ├── test_pedidos.py
│   └── ...
├── scripts/
│   ├── seed.py           # dados de exemplo p/ dev
│   └── import_planilhas.py # CLI do ETL (--dry-run)

```

4. Padrão de camadas (siga sempre)

Fluxo de toda feature: **Rota** → **Controller** → **Service** → **Repository** → **(Model/DB)**.

1. **Rota** (`routers/` para JSON, `web/routes/` para HTML): define método, path, dependency de auth/RBAC, e chama o controller.
2. **Controller** (`controllers/`): valida entrada (Pydantic), chama o(s) service(s), monta a resposta (JSON ou render de template). Sem regra de negócio aqui.
3. **Service** (`services/`): **toda a lógica de negócio**. Orquestra repositories, aplica regras (reserva de estoque, limites por perfil, numeração de pedido). Recebe a `Session` por parâmetro.
4. **Repository** (`repositories/`): só acesso a dados — queries SQLAlchemy. Sem regra de negócio.

Crie sempre os arquivos das 4 camadas + schema. Não pule camadas, mesmo em CRUD simples.

Exemplos de referência (estilo a seguir)

Model (`app/models/produto.py`):

```

from sqlalchemy import String, Integer, Numeric, Boolean, ForeignKey
from sqlalchemy.orm import Mapped, mapped_column, relationship
from app.core.database import Base

class Produto(Base):
    __tablename__ = "produtos"
    id: Mapped[int] = mapped_column(primary_key=True)
    codigo: Mapped[str] = mapped_column(String, unique=True, index=True)
    descricao: Mapped[str] = mapped_column(String, index=True)
    categoria_id: Mapped[int | None] = mapped_column(ForeignKey("categorias.id"))

```

```

unidade: Mapped[str] = mapped_column(String, default="UN")
fator_conversao: Mapped[int] = mapped_column(Integer, default=1)
preco_custo: Mapped[float] = mapped_column(Numeric(12, 2), default=0)
preco_venda: Mapped[float] = mapped_column(Numeric(12, 2), default=0)
estoque_minimo: Mapped[int] = mapped_column(Integer, default=0)
estoque_fisico: Mapped[int] = mapped_column(Integer, default=0)
estoque_reservado: Mapped[int] = mapped_column(Integer, default=0)
ativo: Mapped[bool] = mapped_column(Boolean, default=True)
publicar_catalogo: Mapped[bool] = mapped_column(Boolean, default=False)

```

Schema (`app/schemas/produto.py`):

```

from pydantic import BaseModel, ConfigDict

class ProdutoCreate(BaseModel):
    codigo: str
    descricao: str
    preco_venda: float
    estoque_minimo: int = 0
    # ...

class ProdutoRead(BaseModel):
    model_config = ConfigDict(from_attributes=True)
    id: int
    codigo: str
    descricao: str
    estoque_fisico: int
    estoque_reservado: int
    @property
    def disponivel(self) -> int:
        return self.estoque_fisico - self.estoque_reservado

```

Repository (`app/repositories/produto_repo.py`):

```

from sqlalchemy import select, func
from sqlalchemy.orm import Session
from app.models.produto import Produto

class ProdutoRepository:
    def get_by_codigo(self, db: Session, codigo: str) -> Produto | None:
        return db.scalar(select(Produto).where(Produto.codigo == codigo))

    def busca_rapida(self, db: Session, termo: str, limit: int = 20) -> list[Produto]:
        # usa pg_trgm para descrição + match exato/parcial de códigos
        stmt = (
            select(Produto)
            .where(Produto.descricao.op("%")(termo) | Produto.codigo.ilike(f"{termo}%"))
            .order_by(func.similarity(Produto.descricao, termo).desc())
            .limit(limit)
        )
        return list(db.scalars(stmt))

produto_repo = ProdutoRepository()

```

Service (`app/services/estoque_service.py`) — exemplo da regra crítica de estoque:

```

from sqlalchemy.orm import Session
from app.models.produto import Produto
from app.models.movimentacao import MovimentacaoEstoque, TipoMov
from app.core.errors import RegraNegocioError

class EstoqueService:
    def reservar(self, db: Session, produto: Produto, qtd: int, usuario_id: int, pedido_id:
int) -> None:
        disponivel = produto.estoque_fisico - produto.estoque_reservado
        if qtd > disponivel:
            raise RegraNegocioError(f"Estoque insuficiente para {produto.codigo}: disp.
{disponivel}, pedido {qtd}")
        produto.estoque_reservado += qtd
        db.add(MovimentacaoEstoque(
            produto_id=produto.id, tipo=TipoMov.RESERVA, qtd=qtd,
            origem="pedido", ref_id=pedido_id, usuario_id=usuario_id,
            saldo_apos=produto.estoque_fisico - produto.estoque_reservado,
        ))
        # commit é responsabilidade da rota/uow, não do service

estoque_service = EstoqueService()

```

Controller + Rota web (HTMX) (`app/web/routes/estoque.py`):

```

from fastapi import APIRouter, Depends, Request
from fastapi.responses import HTMLResponse
from app.deps.db import get_db
from app.deps.auth import require_role
from app.core.templates import templates
from app.repositories.produto_repo import produto_repo

router = APIRouter()

@router.get("/estoque/busca", response_class=HTMLResponse)
def busca_estoque(request: Request, q: str = "", db=Depends(get_db),
    user=Depends(require_role("admin", "vendedor", "financeiro",
"funcionario"))):
    produtos = produto_repo.busca_rapida(db, q) if q else []
    # devolve só o fragmento da tabela (HTMX troca o tbody)
    return templates.TemplateResponse("estoque/_linhas.html",
        {"request": request, "produtos": produtos})

```

5. Convenções

Nomenclatura - Arquivos: `snake_case.py` (`produto_repo.py`, `estoque_service.py`). - Models: classe `PascalCase` singular (`Produto`, `Pedido`, `MovimentacaoEstoque`). - Repositories: classe `XRepository`, instância módulo-singleton `x_repo = XRepository()`. - Services: classe `XService`, instância `x_service = XService()`. - Schemas Pydantic: `XCreate`, `XUpdate`, `XRead`. - Rotas: `router = APIRouter()`; funções com nome de ação (`criar_pedido`, `busca_estoque`). - Templates: páginas `modulo/pagina.html`; fragmentos HTMX começam com `_` (`_linhas.html`).

Imports - Sempre absolutos a partir de `app.` (`from app.services.estoque_service import estoque_service`). - `from __future__ import annotations` no topo quando ajudar com type hints.

Erros — exceções de domínio em `app/core/errors.py` (`RegraNegocioError`, `NaoEncontradoError`, `PermissaoNegadaError`) tratadas por um `exception_handler` global que devolve JSON nas rotas de API e um fragmento/flash de erro nas rotas web. **Nunca** vaziar stack trace para o usuário.

Transações / Unit of Work — a `Session` vem do `get_db` (uma por request). Services **não fazem commit**; quem fecha a transação é a rota/controller no fim do fluxo (ou um helper `with uow(db): ...`). Em operações de estoque, garantir atomicidade: ou tudo aplica, ou nada.

Tipagem — type hints em tudo. Ruff configurado para checar imports e estilo.

Dinheiro — `Numeric(12, 2)` no banco; nunca `float` para cálculo financeiro (usar `Decimal`).

6. Modelo de dados (entidades e regras)

Entidades principais (detalhe completo no documento de requisitos, seção 3):

`usuarios`, `categorias`, `fornecedores`, `produtos`, `produto_codigos_empresa` (N códigos de empresa por produto), `produto_imagens` (Fase 2), `clientes`, `pedidos`, `pedido_itens`, `movimentacoes_estoque` (append-only), `inventarios` / `inventario_itens`, `contas_receber`, `auditoria`, `sync_outbox` (Fase 2).

Regras invioláveis de modelagem: - **Estoque nunca é editado direto.** `estoque_fisico` / `estoque_reservado` são saldos materializados, sempre derivados de uma `movimentacoes_estoque` (tipos: entrada, saída, ajuste, reserva, estorno). Toda alteração gera movimentação imutável com usuário, data/hora, origem e `saldo_apos`. - **`produto_codigos_empresa` em tabela própria desde o dia 1** (mesmo que hoje seja 1:1) — confirmar Q1 do cliente. - **Numeração de pedido** sequencial e sem buracos visíveis (sequence dedicada no Postgres). - **`pedidos.origem`** já prevê `local` | `catalogo` | `whatsapp` (Fase 2). - Índices: **GIN trigram** em `produtos.descricao`; índice em todos os campos de código; compostos em pedidos por (vendedor, data) e (cliente, data). - **Auditoria** (`auditoria`) registra antes/depois (JSONB) em produtos, pedidos, estoque e financeiro.

Ciclo de vida do estoque no pedido: `Confirmar` → reserva (`estoque_reservado += qtd`). `Faturar` → baixa definitiva (`estoque_fisico -= qtd`, `estoque_reservado -= qtd`). `Cancelar` → estorno das reservas. (Validar Q4 do cliente.)

7. RBAC — matriz de permissões

Implementar via dependency `require_role(*roles)`. Perfis: `admin`, `vendedor`, `financeiro`, `funcionario`.

Funcionalidade	Admin	Vendedor	Financeiro	Funcionário
Produtos (CRUD)	✓	?	?	?
Preço de custo / margem	✓	✗	✓	✗
Estoque: entradas de mercadoria	✓	✗	✗	✓
Estoque: ajustes manuais	✓	✗	?	✗ (solicita)
Inventário (contagem)	✓	✗	✗	✓ (Admin aprova)
Pedidos: criar/editar	✓ todos	✓ próprios	?	✗
Separação de pedidos (fila)	✓	✗	✗	✓

Funcionalidade	Admin	Vendedor	Financeiro	Funcionário
Faturar / baixar recebimento	✓	✗	✓	✗
Contas a receber	✓	✗	✓	✗
Clientes (CRUD)	✓	✓	?	✗
Relatórios de vendas	✓ todos	?	✓	✗
Relatórios financeiros	✓	✗	✓	✗
Usuários, config, importações	✓	✗	✗	✗
Log de auditoria	✓	✗	✗	✗

Vendedor e Funcionário **não veem** `preco_custo` — filtrar no schema de resposta/template por perfil.

8. Telas / módulos (mapa de implementação)

Protótipos aprovados (referência visual): login, dashboard (Admin), estoque, novo pedido (Vendedor), fila de separação (Funcionário), financeiro (contas a receber). Implementar como páginas Jinja2:

Módulo	Rota base	Perfis	Destaques
Login	<code>/login</code>	todos	argon2 + JWT (cookie httpOnly)
Dashboard	<code>/</code>	Admin (visão), demais reduzido	KPIs do dia, gráfico 7 dias, alertas de mínimo
Estoque	<code>/estoque</code>	todos (níveis distintos)	busca HTMX instantânea, status visual, movimentações
Produtos	<code>/produtos</code>	Admin CRUD	código + códigos de empresa, categorias
Pedidos	<code>/pedidos</code>	Vendedor/Admin	criação com saldo em tempo real, reserva, impressão
Separação	<code>/separacao</code>	Funcionário	fila de confirmados, conferência item a item
Financeiro	<code>/financeiro</code>	Financeiro/Admin	contas a receber, baixas, exportação XLSX
Clientes	<code>/clientes</code>	Vendedor/Admin	cadastro, condição de pagamento
Relatórios	<code>/relatorios</code>	conforme perfil	vendas, ABC, valorização, export
Importação	<code>/importacao</code>	Admin	upload planilha, preview, relatório de erros
Usuários	<code>/usuarios</code>	Admin	CRUD + reset de senha

Impressão (pedido A4 e lista de separação): gerar HTML com CSS `@media print` ou PDF via `weasyprint`. Exportação XLSX: `openpyxl`.

9. ETL das planilhas (módulo `importer/`)

Maior risco do projeto. Pipeline em 6 etapas (detalhe na seção 4 do planejamento): 1. **Coleta** das planilhas reais (estoque + 1 mês de pedidos + planilhas de fornecedor). 2. **Dicionário de dados** (`docs/dicionario-dados.md`): cada coluna → campo canônico. 3. **Staging**: ler bruto (`openpyxl` / `pandas`) para

tabelas `staging_*` sem transformação destrutiva. 4. **Limpeza/validação**: normalização pt-BR (vírgula decimal), dedup por código, detecção de órfãos/conflitos, estoques negativos. Saída: **relatório de inconsistências em XLSX** para o cliente decidir. 5. **Carga definitiva**: importador **idempotente** com `--dry-run`; gera movimentação tipo `importacao` como saldo inicial (rastreável). 6. **Importador no produto**: tela `/importacao` reaproveita o mesmo motor para entradas recorrentes.

CLI: `uv run python scripts/import_planilhas.py --file caminho.xlsx --dry-run`.

10. Modo aplicativo / PWA (terminais)

O cliente quer abrir "como um programa". Entregar PWA instalável: - `static/manifest.webmanifest`: `name="Estrela Gestão", display="standalone", theme_color="#B98A19", background_color="#211B0F", ícones 192/512`. - `static/sw.js`: service worker **network-first** (sempre busca a versão do servidor; cai para cache só offline). Evita terminal preso em versão antiga. - No go-live, criar atalho em modo app: `chrome --app=https://sistema.local` (ou Edge) com ícone na área de trabalho. - Atualização permanece **centralizada no servidor** — terminais nunca precisam reinstalar.

11. Setup do ambiente

Dependências (uv):

```
uv init
uv add fastapi uvicorn[standard] gunicorn sqlalchemy psycopyg[binary] alembic \
    pydantic pydantic-settings pyjwt "passlib[argon2]" jinja2 python-multipart \
    apscheduler openpyxl weasyprint
uv add --dev pytest httpx pytest-cov ruff
```

`.env.example`:

```
DATABASE_URL=postgresql+psycopyg://estrela:senha@localhost:5432/estrela_gestao
JWT_SECRET=troque-isto
JWT_EXPIRES_MIN=480
ENV=dev
```

Comandos (padronizar no README):

```
uv run uvicorn app.main:app --reload          # dev server
uv run alembic revision --autogenerate -m "msg" # nova migration
uv run alembic upgrade head                    # aplicar migrations
uv run pytest --cov=app                        # testes + cobertura
uv run ruff check . && uv run ruff format .     # lint + format
uv run python scripts/seed.py                  # popular dados de dev
docker compose up -d                           # postgres (+ app/caddy em prod)
```

Tailwind (sem Node): baixar o binário standalone do Tailwind e gerar o CSS:

```
./tailwindcss -i app/static/css/input.css -o app/static/css/output.css --watch
```

12. Git workflow

Seguir **integralmente** a skill `git-workflow`. Resumo operacional: - Branches a partir de `dev`: `dev/feat-*`, `dev/fix-*`, `dev/chore-*`. - **Conventional Commits**: `feat(estoque)`: adicionar reserva na confirmação do pedido. - PR de `dev/*` → `dev` (squash). PR `dev` → `main` (release) + tag SemVer. - Nunca commitar direto em `main` / `dev`. Nunca `WIP`. - Abrir um PR por feature, com descrição (o que muda / por quê / como testar).

13. Testes e Definition of Done

Testes (pytest): priorizar os fluxos críticos — - Estoque: reserva, baixa, estorno, ajuste; saldo nunca fica negativo; movimentação sempre criada. - Pedido: total/desconto, limite de desconto por perfil, numeração sem buraco. - RBAC: cada perfil acessa só o que deve (testar 403 nos bloqueios). - Importador: idempotência (rodar 2x não duplica), detecção de inconsistências. - `conftest.py`: banco de teste isolado (transação com rollback por teste) + fixtures de usuário por perfil.

Uma feature só está "pronta" quando: - [] 4 camadas + schema implementadas no padrão. - [] Migration criada e aplicável (`alembic upgrade head` limpo). - [] Testes dos caminhos críticos passando. - [] RBAC aplicado e testado. - [] `ruff check` e `ruff format` limpos. - [] Sem `print` de debug; erros tratados pelo handler global. - [] Tela/HTMX funcionando (quando aplicável) e exportação/impressão se previstas.

14. Roadmap de sprints (execução)

Sprint	Foco	Entregas
S0	Descoberta	Receber planilhas; <code>dicionario-dados.md</code> ; responder Q1-Q16; modelo de dados aprovado; wireframes confirmados
S1	Fundação	Projeto (uv) + Docker + Postgres; <code>core</code> (config/db/security); auth + RBAC (login, cookie JWT, <code>require_role</code>); CRUD produtos/clientes/usuários; busca rápida (<code>pg_trgm</code>); base de templates (layout, sidebar, paleta); importador v1 (staging + relatório de inconsistências)
S2	Estoque	Movimentações (append-only); entradas; ajustes; inventário (contagem + aprovação); alertas de mínimo; posição em tempo real; tela <code>/estoque</code> com busca HTMX e histórico
S3	Pedidos	Fluxo completo (rascunho→confirmado→separação→faturado→entregue→cancelado); reserva/baixa/estorno; descontos com limite por perfil; impressão (pedido A4 + lista de separação); fila <code>/separacao</code> do Funcionário
S4	Financeiro + Relatórios	Contas a receber na fatura; baixas (Pix/boleto/dinheiro); inadimplência; relatórios (vendas, ABC, valorização) + export XLSX; dashboard com KPIs e gráfico
S5	Go-live	PWA + service worker + ícones; empacotar com Docker no servidor local; migração definitiva dos dados; backup (<code>pgBackRest</code>) + Tailscale + monitoramento; configurar modo aplicativo nos 10 terminais ; treinamento por perfil; 1 semana de operação assistida
Fase 2	Catálogo + WhatsApp	Expor services como API JSON; <code>sync_outbox</code> ; catálogo web + imagens (Bunny.net/CDN); Omni Resposta (WhatsApp IA)

15. Primeiros passos (faça nesta ordem)

Quando eu disser para começar o S1, execute na sequência:

1. `uv init` + adicionar dependências (§11) + configurar **Ruff** no `pyproject.toml`.
2. Criar a **estrutura de pastas** da §3 (com `__init__.py`).
3. `docker-compose.yml` com **Postgres 16** + volume; subir e validar conexão.
4. `app/core/`: `config.py` (Settings), `database.py` (engine sync + `SessionLocal` + `Base`), `security.py` (argon2 + JWT).
5. `app/core/errors.py` + handlers globais no `main.py`.
6. Models base: `usuario`, `categoria`, `fornecedor`, `produto`, `produto_codigos_empresa`, `cliente`.
7. **Alembic** init + primeira migration (`autogenerate`) + habilitar extensão `pg_trgm` + índice GIN trigram em `produtos.descricao`.
8. Auth: login (`/login`), hash de senha, emissão de JWT em cookie `httpOnly`; `deps/auth.py` com `get_current_user` e `require_role`.
9. Layout base (Jinja2): `templates/base.html` com sidebar e paleta da marca; incluir HTMX e Alpine.
10. Primeiro CRUD ponta a ponta (**Produtos**) nas 4 camadas + tela com **busca HTMX** — serve de molde para o resto.
11. `scripts/seed.py` com um usuário de cada perfil + alguns produtos.
12. Importador v1 em `importer/` + `scripts/import_planilhas.py --dry-run` (quando as planilhas chegarem).

Ao final do S1, **criar uma skill do projeto** (`estrela-gestao-api`) nos moldes da `profissao-laser-api`, documentando estes padrões para acelerar os próximos sprints.

16. Regras críticas (não errar)

- **Offline-first**: nada na Fase 1 pode depender de internet. Sem CDNs externos em runtime — HTMX, Alpine, Tailwind e fontes servidos localmente de `static/`.
- **Estoque só muda via movimentação** (append-only). Nunca `UPDATE` direto de saldo sem registrar movimentação.
- **Atomicidade** em reserva/baixa/estorno: transação fecha tudo ou nada.
- **Numeração de pedido** sem buracos (sequence Postgres).
- `preco_custo` **oculto** para Vendedor e Funcionário (no schema e no template).
- **Decimal** para dinheiro, nunca float.
- **Auditoria** em produtos, pedidos, estoque e financeiro.
- **Senhas** com argon2; **JWT** em cookie `httpOnly`; sistema **não exposto à internet** (acesso externo só via Tailscale).
- **Service worker network-first** para o PWA não prender versão antiga.
- Mensagens, labels e textos de UI em **português (BR)**.

00 — Índice e Guia do Projeto · Estrela Gestão

Sistema de Estoque e Pedidos — Estrela América do Sul Backend **Python / FastAPI** · interface **Jinja2 + HTMX + Tailwind** · PostgreSQL 16 · 100% local/offline. Este pacote de documentos é o plano de execução para um time inteiro. Cada arquivo cobre uma área.

Como usar estes documentos

Cada `.md` é um **marco** (milestone) autocontido, escrito para ser entregue a uma pessoa ou dupla. A ordem dos números é a ordem recomendada de execução, mas vários rodam em paralelo após o marco 02.

Doc	Marco	Responsável sugerido	Depende de
00-indice-e-guia.md	Visão geral e papéis	Tech Lead	—
01-regras-de-negocio.md	Regras do cliente (áudios + planilha)	Todos leem	—
02-arquitetura-e-fundacao.md	Stack, estrutura, padrões, setup	Tech Lead / Backend Sr	01
03-modelo-de-dados-e-migrations.md	Schema, entidades, migrations	Backend (DB)	01, 02
04-etl-importacao-planilhas.md	Importar a planilha real	Backend (Dados)	01, 03
05-modulo-estoque.md	Estoque, movimentações, localização	Backend (Estoque)	03
06-modulo-pedidos-e-financeiro.md	Pedidos, separação, contas a receber	Backend (Pedidos)	03, 05
07-frontend-ui-pwa.md	Telas, HTMX, modo aplicativo nos terminais	Frontend	02
08-infra-deploy-e-go-live.md	Servidor local, Docker, backup, go-live	DevOps	02

O documento `CLAUDE.md` (entregue à parte) é o contexto-raiz que o agente Claude Code lê. Estes marcos detalham cada frente; o `CLAUDE.md` resume os padrões transversais.

Divisão de trabalho recomendada (time de 3–4 + Claude Code)

- **Tech Lead** — marcos 02 e 08; revisa PRs; mantém `CLAUDE.md` e a skill do projeto.
- **Eduardo (dev principal)** — execução no dia a dia com Claude Code: marcos 03 → 05 → 06.

- **Dev de dados** — marco 04 (ETL), o mais sensível; trabalha junto do cliente nas planilhas.
 - **Frontend** — marco 07 (telas Jinja2/HTMX, PWA), em paralelo a partir do fim do 02.
 - **Tobias** — code review (padrões dos marcos 02 e 01) e testes.
-

Princípios inegociáveis (valem para todos)

1. **Offline-first.** Nada na Fase 1 depende de internet. Sem CDN em runtime — HTMX, Alpine, Tailwind e fontes locais.
 2. **Estoque só muda via movimentação** (append-only). Saldo é sempre derivado, nunca editado direto.
 3. **4 camadas sempre:** Rota → Controller → Service → Repository. Sem pular camada.
 4. **Dinheiro em `Decimal / Numeric(12,2)`**, nunca `float`.
 5. **RBAC** em toda rota (`require_role`). Vendedor e Funcionário não veem custo.
 6. **Tudo em português (BR)** na UI; código e commits seguem convenção (Conventional Commits).
 7. **Conventional Commits + PR por feature** (skill `git-workflow`).
-

Estado do levantamento

✓ **Já temos:** a planilha real (`CONTROLE.xlsx`) e 3 áudios do cliente com as regras de operação — destrinchados no doc 01. **Ainda pendente do cliente:** confirmação das perguntas abertas (listadas no fim do doc 01), principalmente preço por faixa de quantidade e o significado exato de "muito/pouco".

Glossário rápido

- **SKU / código** — código interno do produto (ex.: `K708`, `JSC1140`).
- **Código alternativo** — código da caixa/fábrica secundária do mesmo produto (ex.: Stanley, canetas).
- **Variação** — combinação produto + cor (cada cor tem saldo próprio).
- **Localização** — andar/lado/sala onde o produto fica no estoque físico (10 andares).
- **Movimentação** — registro imutável de qualquer mudança de estoque.
- **Reserva** — estoque comprometido por um pedido confirmado, ainda não faturado.

01 — Regras de Negócio · Estrela Gestão

Fonte: 3 áudios do cliente + análise da planilha real `CONTROLE.xlsx`. **Todos do time leem este documento antes de codar.** Aqui está como a Estrela realmente opera — é o que o sistema precisa respeitar para ser adotado de verdade.

1. Como o cliente opera hoje (planilha `CONTROLE.xlsx`)

A planilha tem **14 abas** divididas em três grupos:

A) Catálogo de estoque (por categoria): `CANETAS PLÁSTICAS`, `CANETAS METÁLICAS`, `COPOS E GARRAFAS`, `BLOCOS E CADERNOS`, `CHAVEIROS`, `ELETRÔNICOS`, `OUTROS`. ~88 produtos distribuídos.

B) Pedidos por cliente: `LUCIANO`, `CLAUDEMIR`, `LEONARDO` (abas com histórico de pedidos), `EM ABERTO` (pedidos/pagamentos pendentes), `NOTAS` (notas fiscais de fornecedor / entradas).

C) Apoio: `COLETA` (dados da empresa para coleta/envio), `BOVESPA` (anotações soltas).

Estrutura de cada produto no catálogo (blocos, não linhas)

Cada produto é um **bloco** com cabeçalho repetido `CÓDIGO` | `DESCRIÇÃO` | `COD ALTERNATIVO` | `CORES` | `QUANTIDADE`:

CÓDIGO	DESCRIÇÃO	COD ALT	CORES	QUANTIDADE	
K708	CANETA ESFEROGRÁFICA COM SUPORTE		BRANCO	156	
			PRETO	212	
1.2			LARANJA	14	← "1.2" = preço (R\$ 1,20)
			VERDE	42	
			AZUL	180	
		...	...	...	2.000 EM CADA CAIXA (nota)

Observações que viram **requisito**: - O **código** aparece uma vez; a **descrição** ocupa várias linhas (nome + detalhe + medida + andar). - Um número solto na coluna `CÓDIGO` abaixo do código (ex.: `1.2`, `0.9`, `16`, `15`) é **preço unitário**. Quando há dois (ex.: `1.8` e `1.6`), são os **dois preços** (pouca vs. muita quantidade) — ver §3. - As **cores são empilhadas**, cada uma com sua **quantidade própria**. - A coluna `QUANTIDADE` mistura **número exato** (`156`), **aproximação** (`tem`, `TEM`, `TEM MUITO`, `TEM POUCO`) e **vazio/zerado** (`-`, `ACABOU`). Ver §2. - Notas livres aparecem em colunas à direita:

`2.000 EM CADA CAIXA`, adicionado ao catálogo, `2º andar`, `3º` e `4º andar` à direita, chega em abril, `PEDIDOS MUITO GRANDES`.

Estrutura de cada pedido (abas de cliente)

2024-11-01	claudemir	241464			← data, cliente, nº do pedido
CODIGO	DESCRICAO	QUANT.	V.UNIT	SUB.TOTAL	
FA12	KIT CHURRASCO	40	19	760	
JSC1140	STANLEY AZUL MARINHO	225	16	3600	← mesmo SKU, cor no nome

JSC1140	STANLEY LARANJA	280	13.9	3892	← preço varia por cor/quantidade
...		TOTAL		9852	

Observações que viram **requisito**: - O pedido referencia **SKU + cor** (a cor está embutida na descrição). - **O mesmo SKU sai com preços unitários diferentes** no mesmo pedido (Stanley a 13,9 / 15,5 / 16), conforme cor e quantidade negociada. O sistema **deve permitir preço unitário editável por item**. - Cada pedido tem **número** (ex.: 241464), data e cliente.

2. Estoque: contagem aproximada → exata (Áudio 1)

"A gente acaba sabendo de cabeça... coloco 'tem muito', 'tem pouco'... só de olhar as pilhas já sei. Mas aí a gente alinha e coloca a quantidade exata, e conforme vai saindo as vendas a gente vai abatendo."

Requisitos: - O produto pode ter estoque em **modo aproximado** (tem muito, tem pouco, tem, acabou) **ou exato** (número). - O sistema deve suportar **os dois estados** por variação: - Campo `estoque_modos`: EXATO | APROXIMADO. - Quando APROXIMADO, guardar um rótulo (MUITO, POUCO, TEM, ACABOU) — sem número confiável. - Quando EXATO, guardar o número e **abater automaticamente a cada venda**. - A **migração inicial** entra com o que a planilha tem (muitos virão como aproximado). A contagem exata é feita aos poucos via **inventário** (doc 05): ao contar, a variação **vira** EXATO e passa a abater. - Na UI, mostrar aproximados com selo (ex.: "tem muito" / "tem pouco" / "acabou") e exatos com o número. - **Importante:** "muito/pouco" **não é** estoque mínimo. É o que o operador enxerga antes de contar. Confirmar com o cliente se querem converter isso em faixas (ex.: pouco = abaixo de X) — Q aberta no §6.

3. Preços: atacado com 2 (às vezes 3) faixas por quantidade (Áudio 2)

"A gente só vende atacado... um preço pra uma caixa só, e outro pra muita quantidade. Coloco 'pouca quantidade' ou 'muita quantidade'. Oficialmente dois preços."

Requisitos: - Cada produto tem **até 3 níveis de preço de venda** por faixa de quantidade: - `preco_pouca_qtd` (ex.: 1 caixa) e `preco_muita_qtd` (volume) — **obrigatórios os dois**. - `preco_promocional` opcional (o "às vezes três"). - O **limite que separa as faixas** (a partir de quantos cai no preço de "muita quantidade") precisa ser **configurável por produto** (campo `qtd_corte_atacado`) — confirmar valor-padrão com o cliente. - No **pedido**, ao informar a quantidade do item, o sistema **sugere** o preço da faixa correspondente, mas o **vendedor pode editar o preço unitário** (a planilha mostra negociação fina por cor). - Vender é **sempre atacado**; não há varejo unitário. - A **margem** é calculada sobre `preco_custo` (vem das notas de fornecedor — aba NOTAS); custo **não aparece** para Vendedor/Funcionário.

4. Cadastro de produtos, SKUs e localização (Áudio 3)

"Cada mês chegam produtos novos... preciso alimentar novos SKUs. Código, descrição, código alternativo (a caixa vem com outro código quando troca de fábrica — copo Stanley, canetas), as cores, quantas unidades vêm em cada caixa (75 copos por caixa → vendo 3 caixas), e ONDE FICA."

Quero um tablet lá embaixo pros funcionários: são 10 andares de estoque e eles se perdem — 4º andar, lado direito, sala 2."

Campos obrigatórios do produto (confirmados pelo cliente): | Campo | Origem na planilha | Observação | |---|---|---| | **Código (SKU)** | coluna CÓDIGO | único, interno | | **Código alternativo** | coluna COD ALTERNATIVO | da caixa/fábrica secundária; pode existir ou não | | **Descrição** | coluna DESCRIÇÃO (multi-linha) | nome + detalhes + medida | | **Categoria** | a aba de origem | Canetas plásticas, Copos, etc. | | **Cores** | coluna CORES (empilhada) | cada cor é uma **variação** com saldo próprio | | **Unidades por caixa** | nota "2.000 EM CADA CAIXA", "75 copos" | `unidades_por_caixa` — permite vender por caixa | | **Localização física** | hoje na descrição ("2º andar", "4º andar dir., sala 2") | **campo próprio** `localizacao` | | **Preço(s)** | número solto na col. CÓDIGO | pouca/muita qtd (ver §3) | | **Quantidade** | coluna QUANTIDADE | por variação; exato ou aproximado (ver §2) |

Requisitos-chave: - **Cadastro de novos SKUs é mensal e frequente** → tela de produto rápida e tela de importação recorrente. - **Localização física é destaque:** precisa ficar **muito visível e pesquisável**. Haverá um **tablet no estoque** para os funcionários consultarem "onde fica o produto X" → tela de consulta de localização simples, grande, busca por código/descrição/cor, mostrando **andar / lado / sala**. (Detalhe de UI no doc 07.) - **Venda por caixa:** como vêm N unidades por caixa, o pedido pode ser em caixas → o sistema converte para unidades no estoque (`unidades_por_caixa`). Mostrar os dois (ex.: "3 caixas = 225 un"). - **Código alternativo** deve ser **pesquisável** (o funcionário pode procurar pelo código da caixa).

5. Resumo das regras que viram comportamento do sistema

1. Produto tem **variações por cor**, cada uma com saldo próprio (exato ou aproximado).
2. Estoque começa aproximado e vira exato via inventário; exato **abate automático** nas vendas.
3. **Dois preços de atacado** por produto (pouca/muita qtd), com corte configurável; **3º preço** opcional; **preço unitário editável no item** do pedido.
4. Produto guarda **código alternativo**, **unidades por caixa** e **localização física** (campo próprio).
5. **Consulta de localização** para tablet no estoque — busca rápida, visual, mostra andar/lado/sala.
6. Pedido referencia **SKU + cor**, permite **venda por caixa**, mesmo SKU com preços diferentes por linha.
7. Cadastro mensal de novos SKUs → cadastro rápido + importação recorrente de planilha.
8. Margem sobre custo (das notas de fornecedor); **custo oculto** para Vendedor/Funcionário.

6. Perguntas abertas para o cliente (confirmar antes de fechar a modelagem)

#	Pergunta	Impacta
Q1	"Tem muito / tem pouco" deve virar faixa numérica (ex.: pouco = abaixo de X) ou continuar só rótulo visual?	Estoque aproximado
Q2	Qual a quantidade de corte padrão entre "pouca" e "muita" quantidade? É por produto ou geral?	Preços
Q3	Vendas saem em caixa , em unidade , ou ambos? Pode vender caixa fracionada?	Pedido/estoque
Q4	A cor é sempre obrigatória no item do pedido? Há produto sem cor?	Variações
Q5	Reserva o estoque ao confirmar o pedido ou só abate ao faturar ?	Estoque

#	Pergunta	Impacta
Q6	Emite Nota Fiscal pelo sistema ou continua emitindo fora? (módulo opcional, R\$ 400/mês)	Escopo
Q7	Localização: o padrão é andar + lado + sala ? Existe prateleira/corredor?	Campo localização
Q8	Os pedidos em aberto (aba EM ABERTO) viram contas a receber no sistema?	Financeiro
Q9	Quantos clientes ativos e quantos terão acesso ao sistema (perfil Vendedor)?	RBAC/licenças
Q10	Confirmar a lista de categorias definitiva (as 7 abas de hoje).	Cadastro

Estas Q substituem/expandem as Q1–Q16 do planejamento anterior, agora ancoradas nos áudios e na planilha real.

02 — Arquitetura e Fundação · Estrela Gestão

Responsável: Tech Lead / Backend Sênior. Entregar primeiro — é a base que todos os outros marcos usam. Espelha o `CLAUDE.md` ; aqui com mais detalhe de decisões e do scaffolding inicial.

1. Stack definitiva

Camada	Tecnologia	Observação
Linguagem	Python 3.12	
Pacotes	uv (Astral)	<code>uv add</code> , <code>uv run</code>
Web/API	FastAPI	
ASGI	Uvicorn (dev) / Gunicorn + UvicornWorker (prod)	
Banco	PostgreSQL 16	extensão <code>pg_trgm</code>
ORM	SQLAlchemy 2.0 (sync) + psycopg 3	sync, não async — §2
Migrations	Alembic	
Schemas	Pydantic v2 + pydantic-settings	
Auth	PyJWT + passlib[argon2]	RBAC via dependencies
UI	Jinja2 + HTMX + Alpine.js + Tailwind	server-rendered
PWA	manifest + service worker network-first	modo aplicativo nos terminais
Agendamento	APScheduler	jobs internos
Impressão/Export	WeasyPrint (PDF) + openpyxl (XLSX)	pedidos, separação, relatórios
Testes	pytest + httpx + pytest-cov	
Lint/format	Ruff	
Container	Docker + Docker Compose	
Proxy	Caddy	HTTPS interno
Manutenção remota	Tailscale	

2. Decisões de arquitetura (e por quê)

Sync, não async. Volume baixo (10 usuários) e transações críticas de estoque (reserva/baixa) pedem previsibilidade. SQLAlchemy sync evita pegadinhas de sessão/transação async. FastAPI roda endpoints `def` em threadpool — performance sobra.

PostgreSQL puro, sem Supabase self-hosted. Menos partes móveis numa máquina que roda sozinha no cliente por meses. Supabase só volta na Fase 2 (cloud).

Jinja2 + HTMX, não SPA. Stack única em Python, sem build de JS, server-rendered (latência mínima na rede local). HTMX entrega busca instantânea e atualização parcial de tabela. A camada **service** fica pronta para expor JSON na Fase 2 (catálogo/WhatsApp).

Estoque append-only. `estoque_fisico` / `estoque_reservado` são saldos materializados, sempre derivados de `movimentacoes_estoque`. Garante auditabilidade total (doc 05).

Offline-first radical. Sem CDN em runtime: HTMX, Alpine, Tailwind e fontes servidos de `static/`.

3. Estrutura do projeto

```
estrela-gestao/
├── CLAUDE.md
├── pyproject.toml
├── .env.example
├── docker-compose.yml
├── docker-compose.prod.yml
├── Dockerfile
├── Caddyfile
├── alembic.ini
├── alembic/versions/
├── app/
│   ├── main.py
│   ├── core/          # config, database, security, errors, templates
│   ├── deps/          # db (get_db), auth (get_current_user, require_role)
│   ├── models/        # SQLAlchemy ORM (1 arquivo por agregado)
│   ├── schemas/       # Pydantic
│   ├── repositories/
│   ├── services/
│   ├── controllers/
│   ├── routers/       # endpoints JSON (Fase 2)
│   ├── web/
│   │   ├── routes/    # rotas que devolvem HTML
│   │   └── templates/ # base, páginas, fragmentos *.html
│   ├── static/        # css, js, icons, manifest.webmanifest, sw.js
│   ├── importer/      # ETL planilhas
│   ├── tests/
│   └── scripts/       # seed.py, import_planilhas.py
```

4. Padrão de 4 camadas (obrigatório)

Fluxo: **Rota** → **Controller** → **Service** → **Repository** → **Model/DB**.

- **Rota** (`web/routes/` HTML, `routers/` JSON): método, path, dependency de auth/RBAC, chama controller.
- **Controller**: valida entrada (Pydantic), chama service(s), monta resposta (render template ou JSON).
- **Service**: **toda regra de negócio**. Recebe `Session`. **Não faz commit** (quem fecha é a rota/UoW).
- **Repository**: só queries SQLAlchemy.

Exemplos completos de cada camada estão no `CLAUDE.md` (§4). Seguir aquele estilo à risca.

Convenções (resumo — detalhe no `CLAUDE.md` §5): arquivos `snake_case`; models `PascalCase` singular; repos/services como singletons de módulo (`produto_repo`, `estoque_service`); schemas `XCreate/XUpdate/XRead`; imports absolutos a partir de `app.`; erros de domínio em `core/errors.py` com handler global; `Decimal` para dinheiro; type hints em tudo.

5. Setup inicial (passo a passo do scaffolding)

```
# 1. Projeto e dependências
uv init estrela-gestao && cd estrela-gestao
uv add fastapi "uvicorn[standard]" gunicorn sqlalchemy "psycopg[binary]" alembic \
    pydantic pydantic-settings pyjwt "passlib[argon2]" jinja2 python-multipart \
    apscheduler openpyxl weasyprint
uv add --dev pytest httpx pytest-cov ruff

# 2. Estrutura de pastas (criar com __init__.py)
mkdir -p app/{core,deps,models,schemas,repositories,services,controllers,routers} \
    app/web/{routes,templates} app/static/{css,js,icons} app/importer tests scripts

# 3. Postgres via Docker
docker compose up -d db

# 4. Alembic
uv run alembic init alembic
# editar alembic/env.py para usar Base e DATABASE_URL do settings

# 5. Primeira migration (após criar os models base — doc 03)
uv run alembic revision --autogenerate -m "schema inicial"
uv run alembic upgrade head

# 6. Rodar
uv run uvicorn app.main:app --reload
```

`pyproject.toml` — configurar Ruff:

```
[tool.ruff]
line-length = 100
target-version = "py312"
[tool.ruff.lint]
select = ["E", "F", "I", "UP", "B"]
```

`.env.example`:

```
DATABASE_URL=postgresql+psycopg://estrela:senha@localhost:5432/estrela_gestao
JWT_SECRET=troque-isto
JWT_EXPIRES_MIN=480
ENV=dev
```

`docker-compose.yml` (dev — só o banco):

```
services:
  db:
    image: postgres:16
    environment:
      POSTGRES_USER: estrela
      POSTGRES_PASSWORD: senha
      POSTGRES_DB: estrela_gestao
    ports: ["5432:5432"]
    volumes: ["pgdata:/var/lib/postgresql/data"]
volumes: { pgdata: {} }
```

6. Bootstrap do app (`app/main.py`)

Responsabilidades mínimas: - Criar o `FastAPI`. - Montar `StaticFiles` em `/static`. - Incluir routers (`web/routes/*` e `routers/*`). - Registrar **exception handlers globais** (`core/errors.py`): domínio → 4xx amigável (JSON nas rotas API, fragmento/flash nas rotas web); inesperado → 500 sem stack trace. - Middleware de autenticação por cookie (lê JWT, injeta usuário). - `core/templates.py`: instância única de `Jinja2Templates(directory="app/web/templates")`.

7. Definition of Done do marco 02

- [] Projeto roda (`uvicorn`) e responde num healthcheck (`/health`).
- [] Postgres sobe via Docker e o app conecta.
- [] `core/` completo: config, database (engine sync + SessionLocal + Base), security (argon2 + JWT), errors, templates.
- [] `deps/` com `get_db` e esqueleto de `require_role`.
- [] Alembic configurado e primeira migration aplicável.
- [] Ruff limpo; estrutura de pastas criada; README com os comandos.
- [] Layout base Jinja2 (`base.html`) com a paleta da marca e HTMX/Alpine locais carregando.

03 — Modelo de Dados e Migrations · Estrela Gestão

Responsável: Backend (DB). Depende dos docs 01 (regras) e 02 (fundação). Este modelo já incorpora os áudios e a planilha real: variações por cor, estoque exato/aproximado, dois preços de atacado, código alternativo, unidades por caixa e localização física.

1. Visão geral das entidades

```
usuarios
categorias
fornecedores
produtos —< produto_variacoes (cor + saldo próprio)
           |< produto_codigos_alt (código alternativo / da caixa)
           |< produto_imagens (Fase 2)
clientes —> pedidos —> pedido_itens
movimentacoes_estoque (append-only; liga a produto_variacao)
inventarios —> inventario_itens
contas_receber
auditoria
sync_outbox (Fase 2)
```

A grande diferença para um catálogo comum: **o saldo de estoque vive na VARIAÇÃO (cor)**, não no produto. O produto guarda os dados comerciais (preços, caixa, localização, códigos); a variação guarda cor e estoque.

2. Tabelas (campos e regras)

produtos

Campo	Tipo	Observação
id	PK	
codigo	str único, index	SKU interno (ex.: K708 , JSC1140)
descricao	text	nome + detalhes (multi-linha vira texto)
categoria_id	FK categorias	
unidades_por_caixa	int	ex.: 2000, 75 — permite venda por caixa
localizacao	str, index	"4º andar, lado direito, sala 2"
preco_pouca_qtd	Numeric(12,2)	1º preço (caixa avulsa)
preco_muita_qtd	Numeric(12,2)	2º preço (volume)
preco_promocional	Numeric(12,2) null	3º preço opcional

Campo	Tipo	Observação
qtd_corte_atacado	int null	a partir de quanto aplica <code>preco_muita_qtd</code>
<code>preco_custo</code>	Numeric(12,2)	das notas de fornecedor; oculto p/ vendedor
<code>ativo</code>	bool	
<code>publicar_catalogo</code>	bool	Fase 2

`produto_variacoes` ← **saldo de estoque mora aqui**

Campo	Tipo	Observação
<code>id</code>	PK	
<code>produto_id</code>	FK produtos	
<code>cor</code>	str	"BRANCO", "AZUL MARINHO"... (pode ser "ÚNICA" se sem cor)
estoque_modo	enum <code>EXATO</code> / <code>APROXIMADO</code>	ver doc 01 §2
estoque_fisico	int	usado quando <code>EXATO</code>
estoque_reservado	int	comprometido por pedidos
rotulo_aprox	enum null <code>MUITO</code> / <code>POUCO</code> / <code>TEM</code> / <code>ACABOU</code>	usado quando <code>APROXIMADO</code>
<code>estoque_minimo</code>	int default 0	alerta de reposição (quando exato)
<code>ativo</code>	bool	

Saldo disponível (modo exato) = `estoque_fisico` - `estoque_reservado`. Toda mudança de `estoque_fisico` / `estoque_reservado` gera `movimentacoes_estoque` (append-only).

`produto_codigos_alt`

Campo	Tipo	Observação
<code>id</code>	PK	
<code>produto_id</code>	FK	
<code>codigo_alt</code>	str, index	código da caixa/fábrica secundária (ex.: <code>K-803</code>)
<code>fornecedor_id</code>	FK fornecedores null	de qual fábrica

Tabela própria desde o dia 1 (um produto pode ganhar mais códigos depois). **Pesquisável** junto com o SKU.

categorias

id, nome — semear com as 7 da planilha: Canetas Plásticas, Canetas Metálicas, Copos e Garrafas, Blocos e Cadernos, Chaveiros, Eletrônicos, Outros. (Confirmar lista — Q10 do doc 01.)

fornecedores

id, nome, cnpj, contato — origem da aba NOTAS.

clientes

id, nome, cnpj_cpf, telefone, endereco, condicao_pagto_padrao, limite_credito null, ativo. Origem: abas de cliente (LUCIANO , CLAUDEMIR , LEONARDO ...).

pedidos

Campo	Tipo	Observação
id	PK	
numero	int único (sequence)	sem buracos; ex.: 241464
cliente_id	FK	
vendedor_id	FK usuarios	
status	enum	rascunho/confirmado/separacao/faturado/entregue/cancelado
desconto_total	Numeric(12,2)	
total	Numeric(12,2)	
observacao	text	
origem	enum	local/catalogo/whatsapp (Fase 2)
criado_em	timestamp	

pedido_itens

Campo	Tipo	Observação
id	PK	
pedido_id	FK	
produto_variacao_id	FK	SKU + cor
qtd	int	em unidades (converter de caixa na entrada)
qtd_caixas	int null	se vendido por caixa, registrar
preco_unit	Numeric(12,2)	editável — sugerido pela faixa, ajustável
desconto	Numeric(12,2)	
subtotal	Numeric(12,2)	

Mesmo SKU pode aparecer em vários itens (cores diferentes, preços diferentes) — é esperado.

movimentacoes_estoque (append-only — nunca UPDATE/DELETE)

Campo	Tipo	Observação
id	PK	
produto_variacao_id	FK	
tipo	enum	entrada/saida/ajuste/reserva/estorno
qtd	int	
origem	enum	pedido/inventario/importacao/manual
ref_id	int null	id do pedido/inventário relacionado
usuario_id	FK	
saldo_apos	int	saldo da variação após a operação
motivo	str null	obrigatório em ajuste
criado_em	timestamp	

inventarios / inventario_itens

- `inventarios`: id, status(aberto/aplicado), criado_em, criado_por.
- `inventario_itens`: id, inventario_id, produto_variacao_id, qtd_sistema, qtd_contada.
- Ao **aplicar** (aprovação Admin): gera `ajuste` em movimentações e marca a variação como `EXATO`.

contas_receber

id, pedido_id, parcela, valor, vencimento, status(pendente/pago/atrasado), forma_pagamento, baixado_em null, baixado_por null. Origem futura: aba `EM ABERTO`.

auditoria

id, usuario_id, entidade, entidade_id, acao, antes(JSONB), depois(JSONB), criado_em. Cobrir produtos, pedidos, estoque e financeiro.

sync_outbox (Fase 2)

id, entidade, entidade_id, payload(JSONB), status, tentativas.

3. Índices e extensões (críticos para busca)

- `CREATE EXTENSION IF NOT EXISTS pg_trgm;`
- **GIN trigram** em `produtos.descricao` e em `produtos.localizacao` (busca do tablet).
- Índice em `produtos.codigo`, `produto_codigos_alt.codigo_alt`, `produto_variacoes.cor`.
- Índice composto em `pedidos (vendedor_id, criado_em)` e `(cliente_id, criado_em)`.
- Índice em `movimentacoes_estoque (produto_variacao_id, criado_em)`.

```
CREATE INDEX ix_produtos_descricao_trgm ON produtos USING gin (descricao gin_trgm_ops);
CREATE INDEX ix_produtos_localizacao_trgm ON produtos USING gin (localizacao gin_trgm_ops);
```

4. Numeração de pedidos sem buracos

Usar **sequence** dedicada no Postgres (`CREATE SEQUENCE pedido_numero_seq START 1;`) lida no service ao confirmar/criar. Não derivar de `count(*)` (gera buraco em concorrência/cancelamento).

5. Migrations (Alembic)

- 1 migration por entrega coesa; nome descritivo (`adicionar_variacoes_e_precos`).
 - A migration inicial cria todas as tabelas-núcleo + extensão `pg_trgm` + índices GIN (em `op.execute`).
 - **Nunca** editar migration já aplicada em produção; criar nova.
 - Toda migration tem `upgrade()` e `downgrade()` válidos.
-

6. Definition of Done do marco 03

- [] Todos os models acima criados no padrão SQLAlchemy 2.0 (`Mapped / mapped_column`).
- [] Enums definidos (`estoque_modos`, `rotulo_aprox`, `tipo_mov`, `status_pedido`, `status_conta`).
- [] Migration inicial aplicável (`alembic upgrade head limpo`) com extensão + índices GIN.
- [] Sequence de número de pedido criada.
- [] `scripts/seed.py` popula: 1 usuário por perfil, as 7 categorias, e ~5 produtos com variações, preços e localização (espelhando a planilha) para os outros marcos trabalharem.
- [] Relacionamentos navegáveis (`produto` → `variações`, `pedido` → `itens`) testados num teste rápido.

04 — ETL / Importação das Planilhas · Estrela Gestão

Responsável: Backend (Dados). Depende dos docs 01 (regras) e 03 (modelo). **É o marco mais sensível do projeto.** A planilha real (`CONTROLE.xlsx`) é organizada por blocos, com dados mistos (números, "tem muito", notas em colunas soltas). Este parsing precisa ser cuidadoso e idempotente.

1. Realidade da planilha (o que o parser enfrenta)

A `CONTROLE.xlsx` tem **14 abas** (ver doc 01 §1). Para a **migração de produtos**, importam as 7 abas de catálogo. Cada produto é um **bloco** separado por um cabeçalho repetido:

CÓDIGO	DESCRIÇÃO	COD ALTERNATIVO	CORES	QUANTIDADE	
K708	CANETA ESFEROGRÁFICA		BRANCO	156	
	COM SUPORTE		PRETO	212	
1.2			LARANJA	14	
			VERDE	42	[col. extra: "2.000 EM
	CADA CAIXA"]				

Padrões a tratar (todos vistos na planilha real): - **Código na 1ª linha do bloco**; descrição em **várias linhas** (concatenar). - **Número solto** na coluna CÓDIGO abaixo do código = **preço** (1.2 , 0.9 , 16). Dois números = dois preços (pouca/muita qtd). Heurística: valor com decimal pequeno → preço. - **Cores empilhadas**, cada uma com **quantidade própria**. - **Quantidade mista**: número (156), texto (tem , TEM , TEM MUITO , TEM POUCO), vazio, -, ACABOU. - **Localização** aparece dentro da descrição (2º andar , 3º e 4º andar à direita) → extrair para campo próprio. - **Notas** em colunas à direita (2.000 EM CADA CAIXA , adicionado ao catálogo , chega em abril , PEDIDOS MUITO GRANDES) → extrair `unidades_por_caixa` quando possível; resto vira observação. - **Código alternativo** quando presente na coluna COD ALTERNATIVO (ex.: K-820 → K-803). - Abas têm **dimensões e nº de colunas diferentes** (de 7 a 13 colunas) — parser não pode assumir layout fixo.

Para pedidos (abas de cliente), o ETL é **opcional na Fase 1** (histórico). Estrutura: linha data | cliente | nº , depois CODIGO | DESCRICAO | QUANT | V.UNIT | SUB.TOTAL até TOTAL . Importar só se o cliente quiser histórico migrado — senão começa zerado.

2. Pipeline em 6 etapas

Etapla 1 — Coleta. A planilha já está em mãos (`CONTROLE.xlsx`). Pedir também notas de fornecedor para custo.

Etapla 2 — Dicionário de dados (`docs/dicionario-dados.md`): mapear, por aba, qual coluna é o quê. Já adiantado neste documento; refinar com o cliente nas Q abertas (doc 01 §6).

Etapa 3 — Staging. Ler o bruto para tabelas `staging_produtos` / `staging_variacoes` **sem transformar**. Guardar aba de origem, nº da linha e valor literal — rastreabilidade total.

Etapa 4 — Parsing + validação. O parser de blocos (§3) gera registros canônicos. Validar: - preço presente? (senão, marcar para revisão) - quantidade interpretável? (número → exato; texto conhecido → aproximado; desconhecido → revisão) - código duplicado? código alternativo conflitante? - cor sem quantidade / quantidade sem cor. Saída: **relatório de inconsistências em XLSX** (`openpyxl`) com aba/linha/problema → cliente decide. **Não decidir por ele.**

Etapa 5 — Carga definitiva. Importador **idempotente** com `--dry-run`: - `produtos` (com preços, localização extraída, unidades por caixa); - `produto_variacoes` (cor + estoque: número → EXATO +saldo; texto → APROXIMADO +rótulo); - `produto_codigos_alt`; - movimentação tipo `importacao` como **saldo inicial** das variações exatas (rastreável). Rodar 2x **não duplica** (chave: `codigo` do produto; `produto_id+cor` da variação).

Etapa 6 — Importador recorrente. A tela `/importacao` (doc 07) reaproveita o mesmo motor para as entradas mensais de novos SKUs, com preview e relatório de erros — o cliente não depende mais do time para importar.

3. Esboço do parser de blocos (referência)

```
# app/importer/parser.py
from dataclasses import dataclass, field
import openpyxl, re

ROTULOS = {"TEM MUITO": "MUITO", "TEM POUCO": "POUCO", "TEM": "TEM",
           "ACABOU": "ACABOU", "-": "ACABOU"}

@dataclass
class VariacaoIn:
    cor: str
    qtd_raw: str
    estoque_mod: str = "APROXIMADO"
    estoque_fisico: int = 0
    rotulo: str | None = None

@dataclass
class ProdutoIn:
    codigo: str
    descricao: str
    categoria: str
    cod_alt: str | None = None
    precos: list[float] = field(default_factory=list) # [pouca, muita]
    unidades_por_caixa: int | None = None
    localizacao: str | None = None
    obs: str | None = None
    variacoes: list[VariacaoIn] = field(default_factory=list)

def interpreta_qtd(v) -> VariacaoIn:
    s = str(v).strip().upper() if v is not None else ""
    if s.isdigit():
        return VariacaoIn(cor="", qtd_raw=s, estoque_mod="EXATO", estoque_fisico=int(s))
    if s in ROTULOS:
        return VariacaoIn(cor="", qtd_raw=s, estoque_mod="APROXIMADO", rotulo=ROTULOS[s])
    # número com unidade ("375 UNID"): extrair dígitos
    m = re.search(r"\d+", s)
```

```

    if m:
        return VariacaoIn(cor="", qtd_raw=s, estoque_modos="EXATO",
estoque_fisico=int(m.group()))
    return VariacaoIn(cor="", qtd_raw=s, estoque_modos="APROXIMADO", rotulo="TEM" if s else
None)

def parse_aba(ws, categoria: str) -> list[ProdutoIn]:
    produtos, atual = [], None
    for row in ws.iter_rows(values_only=True):
        c = [(x if x is not None else "") for x in row]
        col_codigo = str(c[0]).strip() if c and c[0] else ""
        # novo bloco começa no header
        if col_codigo == "CÓDIGO":
            atual = None
            continue
        # início de produto: tem código "de verdade" (não só número de preço)
        if col_codigo and not _eh_preco(col_codigo):
            atual = ProdutoIn(codigo=col_codigo, descricao=str(c[1] or "").strip(),
                            categoria=categoria, cod_alt=(str(c[2]).strip() or None) if
len(c) > 2 else None)
            produtos.append(atual)
        elif atual and _eh_preco(col_codigo):
            atual.precos.append(float(col_codigo.replace(",", ".")))
        # descrição extra
        if atual and c[1] and col_codigo == "" :
            atual.descricao = (atual.descricao + " " + str(c[1]).strip()).strip()
            _extraia_localizacao_e_caixa(atual, str(c[1]))
        # cor + quantidade
        cor = str(c[3]).strip() if len(c) > 3 and c[3] else ""
        if atual and cor and cor != "CORES":
            vi = interpreta_qtd(c[4] if len(c) > 4 else None)
            vi.cor = cor
            atual.variacoes.append(vi)
    return produtos

def _eh_preco(s: str) -> bool:
    return bool(re.fullmatch(r"\d{1,3}([.]\d{1,2})?", s)) and "." in s or "," in s or len(s)
<= 2 and s.isdigit()

```

Este esboço é ponto de partida — ajustar com casos reais. **Cobrir com testes** usando trechos da planilha (ver §5). O parser é a parte que mais quebra; testar exaustivamente.

4. CLI

```

uv run python scripts/import_planilhas.py --file CONTROLE.xlsx --dry-run # valida, gera
relatório, não grava
uv run python scripts/import_planilhas.py --file CONTROLE.xlsx # grava
uv run python scripts/import_planilhas.py --file CONTROLE.xlsx --so-categoria "COPOS E
GARRAFAS"

```

Saídas: `relatorio_inconsistencias.xlsx` + log resumo (quantos produtos, variações, exatos vs aproximados, erros).

5. Testes obrigatórios

- Bloco com **dois preços** → produto com `preco_pouca_qtd` e `preco_muita_qtd` corretos.
 - Quantidade `156` → `EXATO /156`; `TEM MUITO` → `APROXIMADO / MUITO`; `- / ACABOU` → `ACABOU`; `375 UNID` → `EXATO /375`.
 - Descrição com `"4º andar à direita"` → `localizacao` extraída, descrição preservada.
 - `"2.000 EM CADA CAIXA"` → `unidades_por_caixa = 2000`.
 - Código alternativo `K-820` → `K-803` → registro em `produto_codigos_alt`.
 - **Idempotência**: importar 2x não duplica produto nem variação.
 - Variações exatas geram movimentação `importacao` com `saldo_apos` correto.
-

6. Definition of Done do marco 04

- [] Parser de blocos lê as 7 abas de catálogo sem perder produtos.
- [] Staging preserva o bruto (aba/linha/valor).
- [] Relatório de inconsistências em XLSX gerado e legível pelo cliente.
- [] Carga idempotente com `--dry-run`; variações exatas com movimentação inicial.
- [] Localização, unidades por caixa, preços e código alternativo extraídos.
- [] Testes do §5 passando.
- [] Mesmo motor exposto para a tela `/importacao` (entrega junto do doc 07 ou logo após).

05 — Módulo de Estoque · Estrela Gestão

Responsável: Backend (Estoque). Depende do doc 03 (modelo). Cobre o coração do sistema: movimentações append-only, estoque exato vs aproximado, inventário, alertas, e a **consulta de localização para o tablet** do estoque.

1. Conceito central: saldo derivado de movimentações

O saldo de uma **variação** (produto + cor) **nunca é editado direto**. Toda mudança passa por um service que: 1. valida a regra, 2. ajusta `estoque_fisico` / `estoque_reservado` da variação, 3. grava uma `movimentacoes_estoque` imutável com `saldo_apos`, usuário, origem e (se ajuste) motivo.

Tipos de movimentação: `entrada`, `saida`, `ajuste`, `reserva`, `estorno`.

```
# app/services/estoque_service.py (núcleo)
class EstoqueService:
    def _registrar(self, db, variacao, tipo, qtd, origem, usuario_id, ref_id=None,
motivo=None):
        db.add(MovimentacaoEstoque(
            produto_variacao_id=variacao.id, tipo=tipo, qtd=qtd, origem=origem,
            ref_id=ref_id, usuario_id=usuario_id, motivo=motivo,
            saldo_apos=variacao.estoque_fisico - variacao.estoque_reservado,
        ))

    def entrada(self, db, variacao, qtd, usuario_id, origem="manual", ref_id=None):
        variacao.estoque_modos = "EXATO" # dar entrada torna o saldo exato
        variacao.estoque_fisico += qtd
        self._registrar(db, variacao, "entrada", qtd, origem, usuario_id, ref_id)

    def reservar(self, db, variacao, qtd, usuario_id, pedido_id):
        if variacao.estoque_modos == "EXATO":
            disp = variacao.estoque_fisico - variacao.estoque_reservado
            if qtd > disp:
                raise RegraNegocioError(f"Estoque insuficiente ({variacao.cor}): disp {disp},
pedido {qtd}")
            variacao.estoque_reservado += qtd
            self._registrar(db, variacao, "reserva", qtd, "pedido", usuario_id, pedido_id)

    def baixar(self, db, variacao, qtd, usuario_id, pedido_id):
        variacao.estoque_fisico -= qtd
        variacao.estoque_reservado -= qtd
        self._registrar(db, variacao, "saida", qtd, "pedido", usuario_id, pedido_id)

    def estornar(self, db, variacao, qtd, usuario_id, pedido_id):
        variacao.estoque_reservado -= qtd
        self._registrar(db, variacao, "estorno", qtd, "pedido", usuario_id, pedido_id)

    def ajustar(self, db, variacao, novo_saldo, usuario_id, motivo):
        if not motivo:
            raise RegraNegocioError("Ajuste exige motivo")
```



```
delta = novo_saldo - variacao.estoque_fisico
variacao.estoque_moda = "EXATO"
variacao.estoque_fisico = novo_saldo
self._registrar(db, variacao, "ajuste", abs(delta), "inventario", usuario_id,
motivo=motivo)
```

Atomicidade: a rota que confirma/fatura um pedido abre uma transação e só faz commit no fim. Se um item falhar a reserva, **nada** é aplicado.

2. Estoque exato vs aproximado (regra do Áudio 1)

- Variação nasce, na migração, geralmente como **APROXIMADO** (rótulo **MUITO** / **POUCO** / **TEM** / **ACABOU**).
- Vira **EXATO** quando: recebe **entrada**, sofre **ajuste**, ou é **contada no inventário**.
- Em **EXATO**, vendas **abatem automaticamente** (reserva → baixa).
- Em **APROXIMADO**, o sistema **não bloqueia** venda por saldo (não há número confiável), mas **avisa** que o estoque não está exato; ideal levar essa variação a um inventário.
- Na listagem, exibir:
- **EXATO**: número + (se ≤ mínimo) selo de alerta.
- **APROXIMADO**: selo  muito /  pouco /  tem /  acabou.

3. Inventário (contagem)

Fluxo (perfil **Funcionário** conta, **Admin** aprova): 1. Funcionário abre inventário (geral, por categoria, ou por produto) → gera `inventario_itens` com `qtd_sistema`. 2. Conta fisicamente e preenche `qtd_contada`. 3. Admin **aplica**: para cada item, `ajustar(novo_saldo=qtd_contada, motivo="inventário #N")`, marcando a variação como **EXATO**. Movimentações **ajuste** são geradas em lote. 4. Inventário fica **aplicado** e auditado.

4. Entradas de mercadoria

- Manual (Admin/Funcionário): escolhe produto+cor, informa quantidade (em unidades **ou caixas** → converter por `unidades_por_caixa`), service faz **entrada**.
- Por importação (recorrente): motor do ETL (doc 04) gera entradas.
- Mostrar conversão na UI: "3 caixas × 75 = 225 unidades".

5. Alertas e reposição

- Variação **EXATO** com `estoque_fisico ≤ estoque_minimo` → entra no **relatório de reposição** e nos alertas do dashboard.
- Aproximados com rótulo **POUCO** / **ACABOU** também aparecem como atenção (sem número).

6. Consulta de localização (tablet do estoque) — destaque do cliente

Áudio 3: "Quero deixar um tablet lá embaixo pros funcionários... são 10 andares e eles se perdem."

Endpoint dedicado, **simples e visual** (UI no doc 07): - Busca por **código, código alternativo, descrição ou cor** (HTMX, instantânea via pg_trgm). - Resultado mostra, **grande**: descrição, cor(es) e **LOCALIZAÇÃO** (andar / lado / sala), além de uma indicação de saldo (número ou selo). - Repository com `busca_rapida` cobrindo `produtos.codigo`, `produto_codigos_alt.codigo_alt`, `produtos.descricao` (trigram) e `produto_variacoes.cor`.

```
def busca_localizacao(self, db, termo, limit=15):
    stmt = (select(Produto)
            .where(
                Produto.codigo.ilike(f"{termo}%")
                | Produto.descricao.op("%")(termo)
                | Produto.localizacao.op("%")(termo)
                |
                Produto.id.in_(select(ProdutoCodigoAlt.produto_id).where(ProdutoCodigoAlt.codigo_alt.ilike(f"{termo}%"))))
            ).limit(limit))
    return list(db.scalars(stmt))
```

7. Endpoints do módulo (resumo)

Rota	Perfil	Função
GET /estoque	todos (níveis)	lista/paginada com busca
GET /estoque/busca?q=	todos	fragmento HTMX da tabela
GET /estoque/localizacao	todos (tablet)	consulta de localização
POST /estoque/entrada	admin, funcionario	dar entrada
POST /estoque/ajuste	admin	ajuste com motivo
GET /inventario / POST /inventario	funcionario (abre), admin (aplica)	contagem
GET /estoque/{variacao}/movimentacoes	admin, financeiro	histórico

8. Definition of Done do marco 05

- [] `EstoqueService` com entrada/reserva/baixa/estorno/ajuste, sempre gerando movimentação.
- [] Saldo nunca negativo em modo `EXATO`; reserva respeita disponível.
- [] Transições `APROXIMADO` → `EXATO` nos pontos certos (entrada/ajuste/inventário).
- [] Inventário completo (abrir, contar, aplicar com aprovação) e auditado.
- [] Conversão caixa↔unidade na entrada.
- [] Consulta de localização rápida e abrangente (código, cód. alt, descrição, cor).
- [] Alertas de mínimo + relatório de reposição.
- [] Testes: reserva/baixa/estorno, ajuste com motivo, idempotência de saldo, busca de localização.

06 — Módulo de Pedidos e Financeiro · Estrela Gestão

Responsável: Backend (Pedidos). Depende dos docs 03 (modelo) e 05 (estoque). Cobre o ciclo do pedido (com preço por faixa e venda por caixa), a fila de separação do Funcionário, e o contas a receber.

1. Ciclo de vida do pedido

```
rascunho → confirmado → separacao → faturado → entregue  
                |  
                → cancelado
```

- **rascunho**: vendedor monta, edita livremente. Não mexe em estoque.
- **confirmado**: **reserva** o estoque de cada item (doc 05). Entra na fila de separação.
- **separacao**: funcionário confere item a item (§3).
- **faturado**: **baixa** definitiva do estoque + gera **contas a receber** (§4).
- **entregue**: encerrado.
- **cancelado**: **estorna** reservas (se estava confirmado/separação). Auditado.

Validar com o cliente (Q5 do doc 01) se a reserva é na confirmação ou só no faturamento. O código acima assume reserva na confirmação (mais seguro para atacado de alto giro).

2. Montagem do pedido (regras dos áudios)

- Item referencia **produto + variação (cor)**. Cor normalmente obrigatória (Q4 do doc 01).
- **Preço por faixa de quantidade** (Áudio 2): ao informar a quantidade, o service **sugere**:
 - `qtd < qtd_corte_atacado` → `preco_pouca_qtd`
 - `qtd ≥ qtd_corte_atacado` → `preco_muita_qtd`
- há `preco_promocional`? oferecer como opção.
- **Preço unitário é editável** no item (a planilha mostra o mesmo SKU saindo a preços diferentes por cor/negociação).
- **Venda por caixa**: vendedor pode informar **caixas**; o sistema converte para unidades (`qtd = qtd_caixas × unidades_por_caixa`) e mostra os dois. Guarda `qtd_caixas` no item.
- **Disponibilidade em tempo real**: ao adicionar item, mostrar saldo da variação (número se `EXATO`, selo se `APROXIMADO`).
- **Desconto** por item e total, com **limite por perfil** (vendedor até X%; acima exige Admin).

```
# app/services/preco_service.py  
def sugerir_preco(produto, qtd) -> Decimal:  
    corte = produto.qtd_corte_atacado or 0  
    if corte and qtd >= corte:  
        return produto.preco_muita_qtd  
    return produto.preco_pouca_qtd
```

```
# app/services/pedido_service.py (confirmar)
def confirmar(self, db, pedido, usuario_id):
    if not pedido.itens:
        raise RegraNegocioError("Pedido sem itens")
    for item in pedido.itens:
        estoque_service.reservar(db, item.variacao, item.qtd, usuario_id, pedido.id)
    pedido.numero = pedido.numero or proximo_numero_pedido(db) # sequence
    pedido.status = "confirmado"
    # commit é da rota
```

3. Fila de separação (perfil Funcionário)

- Tela `/separacao` lista pedidos **confirmados**, em ordem de chegada.
- Ao abrir um pedido, o funcionário vê os itens com **localização** (ajuda a achar no estoque — Áudio 3) e confere um a um (checkbox HTMX). Barra de progresso.
- **Concluir separação** → status `separacao` finalizada (pronto para faturar). Cada conferência é registrada com usuário/data/hora (auditoria).
- Imprimir **lista de separação** (PDF via WeasyPrint) com itens, cores, quantidades e **localização**.

4. Financeiro — contas a receber

- Ao **faturar**, gerar título(s) em `contas_receber` conforme condição de pagamento do cliente (à vista, parcelado, prazo). Origem futura dos pendentes: aba `EM ABERTO`.
- **Baixa de recebimento** (Pix, boleto, dinheiro) com data e usuário → status `pago`.
- Vencido sem baixa → `atrasado` (job diário do APScheduler reavalia).
- Relatórios: recebíveis por período/cliente, inadimplência, recebimentos do dia.
- **Margem** (custo × venda) visível só para Financeiro/Admin.

5. Impressão e exportação

- **Pedido (A4)** e **lista de separação**: HTML com CSS `@media print` ou PDF (WeasyPrint).
- **Relatórios** exportáveis em **XLSX** (openpyxl): vendas por período/vendedor/cliente/produto, curva ABC, valorização de estoque.

6. Endpoints do módulo (resumo)

Rota	Perfil	Função
GET/POST <code>/pedidos</code>	vendedor, admin	listar/criar
POST <code>/pedidos/{id}/itens</code>	vendedor, admin	adicionar item (sugere preço, mostra saldo)
POST <code>/pedidos/{id}/confirmar</code>	vendedor, admin	reserva estoque
POST <code>/pedidos/{id}/cancelar</code>	admin (vendedor: próprio rascunho)	estorna
GET <code>/separacao</code>	funcionario	fila
POST <code>/separacao/{id}/concluir</code>	funcionario	finaliza separação

Rota	Perfil	Função
POST /pedidos/{id}/faturar	financeiro, admin	baixa + contas a receber
GET /financeiro	financeiro, admin	contas a receber
POST /financeiro/{conta}/baixar	financeiro, admin	registrar recebimento
GET /relatorios/*	conforme perfil	relatórios + export

7. Definition of Done do marco 06

- [] Ciclo completo do pedido com reserva/baixa/estorno integrados ao `EstoqueService`.
- [] Sugestão de preço por faixa + **preço unitário editável** no item.
- [] Venda por **caixa** com conversão para unidades.
- [] Numeração de pedido por sequence (sem buracos).
- [] Desconto com limite por perfil.
- [] Fila de separação com conferência item a item + localização + impressão.
- [] Contas a receber na fatura, baixas, status atrasado via job.
- [] Relatórios + export XLSX; impressão de pedido e separação em PDF.
- [] Testes: totais/descontos, faixa de preço, conversão de caixa, ciclo de status, geração de contas a receber.

07 — Frontend, UI e PWA · Estrela Gestão

Responsável: Frontend. Depende do doc 02 (fundação). Roda em paralelo a partir do fim do marco 02. Stack: **Jinja2 + HTMX + Alpine.js + Tailwind**, server-rendered, **100% local** (assets servidos de `static/`).

1. Identidade visual (já aprovada nos protótipos)

Paleta da marca Estrela (estrela dourada): - **Dourado primário** #B98A19 — botões, destaques, barras. - **Dourado escuro** #8C660E — textos de ênfase, valores. - **Creme** #F6F2E8 — fundo geral. - **Sidebar** #211B0F (marrom quase preto) — menu lateral, texto creme #D8CBAC. - **Cartões** brancos, bordas #E4DCCA. - Status: ok/pago verde #1F7A40 (fundo #E3F2E8), atenção âmbar, crítico/atrasado vermelho #B3261E, aguardando azul.

Tipografia: sem serifa, leitura densa de tabelas. Formatação enxuta. Tudo em **português (BR)**.

Definir tokens no `tailwind.config` (cores `gold`, `cream`, `sidebar` ...) e usar utilitárias. Configurar o **Tailwind Standalone CLI** (sem Node) — ver doc 02 §5.

2. Telas (mapa — protótipos de referência existem)

Tela	Rota	Perfil	Notas
Login	/login	todos	card central, logo Estrela, JWT em cookie
Dashboard	/	Admin (cheio)	KPIs do dia, gráfico 7 dias, alertas de mínimo, últimos pedidos
Estoque	/estoque	todos	busca HTMX instantânea, status (número/selo), movimentações
Localização (tablet)	/estoque/localizacao	funcionario	tela grande e simples — ver §4
Produtos	/produtos	admin	SKU + cód. alternativo + cores + preços + localização + caixa
Novo pedido	/pedidos/novo	vendedor, admin	saldo em tempo real, sugestão de preço, venda por caixa
Separação	/separacao	funcionario	fila + conferência item a item + localização
Financeiro	/financeiro	financeiro, admin	contas a receber, baixas, export
Clientes	/clientes	vendedor, admin	cadastro

Tela	Rota	Perfil	Notas
Relatórios	/relatorios	conforme perfil	vendas, ABC, valorização, export XLSX
Importação	/importacao	admin	upload, preview, relatório de erros (motor do doc 04)
Usuários	/usuarios	admin	CRUD + reset de senha

3. Padrões HTMX

- **Busca instantânea:** input com `hx-get="/estoque/busca" hx-trigger="keyup changed delay:250ms" hx-target="#linhas"`. O servidor devolve **só o fragmento** (`_linhas.html`) — troca o `tbody`.
- **Ações** (confirmar pedido, baixar conta, conferir item): `hx-post` devolvendo o fragmento atualizado.
- **Templates:** páginas em `templates/<modulo>/<pagina>.html` estendem `base.html`; fragmentos começam com `_`.
- **Alpine** só para interações locais leves (abrir modal, toggle), sem estado de negócio.
- **Feedback de erro:** o handler global (doc 02) devolve um fragmento de alerta; exibir via `hx-target` de erro ou um container de flash.

Estrutura sugerida de templates:

```
templates/
├── base.html           # layout: sidebar + topbar + bloco de conteúdo
├── _flash.html        # mensagens
├── login.html
├── dashboard.html
├── estoque/
│   ├── index.html
│   ├── _linhas.html   # fragmento da tabela (HTMX)
│   └── localizacao.html # tela do tablet
├── pedidos/
│   ├── novo.html
│   └── _item_linha.html
├── separacao/
│   ├── index.html
│   └── _item_check.html
├── financeiro/index.html
└── ...
```

4. Tela de localização (tablet) — requisito destacado pelo cliente

Áudio 3: 10 andares, funcionários se perdem, querem um tablet no estoque. Esta tela é **especial**: - **Fonte grande**, alto contraste, poucos elementos — usável em pé, num tablet, por funcionário novo. - Um **campo de busca** dominante (código, cód. alternativo, descrição ou cor). - Resultado em **cartões grandes**: descrição + cor + **LOCALIZAÇÃO em destaque** (ex.: "4º ANDAR · LADO DIREITO · SALA 2") + indicação de saldo (número ou selo). - Sem ações de edição — é **consulta**. Pode ter modo "tela cheia/quiosque". - Funciona offline como o resto (PWA).

5. PWA — "abrir como programa" nos terminais

Requisito do cliente: abrir como um programa, não como site. - `static/manifest.webmanifest`:
`name="Estrela Gestão", short_name="Estrela", display="standalone", theme_color="#B98A19",`
`background_color="#211B0F"`, ícones 192/512 (estrela dourada). - `static/sw.js`: service worker
network-first (busca sempre a versão do servidor; cache só fallback offline). Evita terminal preso em
versão antiga. - `base.html` referencia o manifest e registra o service worker. - No go-live (doc 08), criar
atalho `chrome --app=https://sistema.local` com ícone na área de trabalho de cada terminal. -
Atualização **centralizada no servidor**; terminais nunca reinstalam.

6. Acessibilidade e performance

- Tabelas densas com cabeçalho fixo; paginação server-side (estoque tem 12k+ linhas).
 - Atalhos de teclado nas telas de alto volume (novo pedido, busca de estoque).
 - Imagens e ícones locais; nada de fontes/CDN externas (offline-first).
-

7. Definition of Done do marco 07

- [] `base.html` com sidebar/topbar e a paleta da marca; HTMX/Alpine/Tailwind locais.
- [] Login funcional (cookie JWT) e navegação por perfil (menu muda conforme papel).
- [] Estoque com busca HTMX instantânea e status (número/selo).
- [] **Tela de localização (tablet)** grande, simples e rápida.
- [] Novo pedido com saldo em tempo real, sugestão de preço e venda por caixa.
- [] Fila de separação com conferência e impressão.
- [] Financeiro com baixas e export.
- [] PWA instalável (manifest + service worker network-first) — abre em modo aplicativo.
- [] Telas em português, formatação enxuta, paginação server-side onde há volume.

08 — Infraestrutura, Deploy e Go-Live · Estrela Gestão

Responsável: DevOps / Tech Lead. Depende do doc 02 (fundação). O sistema roda **localmente no cliente**, em um **mini PC**, acessado por até 10 terminais via rede interna.

1. Hardware (cenário enxuto definido)

Para 10 usuários e 80–100 pedidos/dia, o workload é leve — o que importa é confiabilidade básica e backup.

Item	Especificação	Faixa (jun/2026)
Mini PC	Ryzen 5/7, 16–32 GB RAM, SSD 512 GB	R\$ 1.800–3.200
Nobreak senoidal 1500 VA	NHS/Intelbras, USB p/ shutdown	R\$ 1.300–1.800
HD externo 1 TB	backup local (rotação)	R\$ 300–450
Switch gigabit 8 portas	rede dos terminais	R\$ 150–280
Total		≈ R\$ 3.500–5.700

Trade-off aceito: sem RAID/ECC. Mitigação: **backup automático diário** + cópia offsite quando houver internet; se o mini PC falhar, compra-se outro e restaura-se o backup em < 2 h.

2. Topologia

```
[10 terminais: navegador em modo app] — switch gigabit — [MINI PC / SERVIDOR]
                                     Caddy (443, HTTPS interno)
                                     └─ App (Gunicorn+Uvicorn, FastAPI)
                                     └─ PostgreSQL 16 (Docker)
                                     Backup diário → HD externo +
offsite
                                     Tailscale → manutenção remota
NextLayer
[Nobreak alimentando mini PC + switch + roteador]
```

- Terminais acessam `https://sistema.local` (DNS do roteador ou mDNS/Avahi). Zero instalação nas estações.
- Mini PC com **IP fixo**, BIOS para **religar após queda de energia**, nobreak com shutdown via USB (NUT).

3. Deploy com Docker Compose (produção)

`docker-compose.prod.yml` com três serviços: `db` (Postgres 16), `app` (imagem do projeto, Gunicorn + UvicornWorker), `caddy` (proxy HTTPS interno).

```

services:
  db:
    image: postgres:16
    environment:
      POSTGRES_USER: estrela
      POSTGRES_PASSWORD: ${DB_PASSWORD}
      POSTGRES_DB: estrela_gestao
    volumes: ["pgdata:/var/lib/postgresql/data"]
    restart: always

  app:
    build: .
    command: gunicorn app.main:app -k uvicorn.workers.UvicornWorker -w 3 -b 0.0.0.0:8000
    environment:
      DATABASE_URL: postgresql+psycopg://estrela:${DB_PASSWORD}@db:5432/estrela_gestao
      JWT_SECRET: ${JWT_SECRET}
      ENV: prod
    depends_on: [db]
    restart: always

  caddy:
    image: caddy:2
    ports: ["443:443", "80:80"]
    volumes:
      - ./Caddyfile:/etc/caddy/Caddyfile
      - caddy_data:/data
    depends_on: [app]
    restart: always

volumes: { pgdata: {}, caddy_data: {} }

```

Caddyfile (HTTPS interno automático para `sistema.local`):

```

sistema.local {
  reverse_proxy app:8000
  tls internal
}

```

Dockerfile: base `python:3.12-slim`, instalar dependências do sistema do WeasyPrint (libpango, libcairo), copiar app, instalar via uv, expor 8000.

4. Atualizações

- Build/tag de versão → no servidor: `docker compose pull && docker compose up -d` (via Tailscale).
- **Rollback** imediato para a tag anterior se algo quebrar.
- Migrations: rodar `alembic upgrade head` no container do app após subir nova versão (entrypoint ou comando manual).
- Janela combinada com o cliente (fora do horário comercial).

5. Backup (regra 3-2-1 adaptada ao enxuto)

Camada	O quê	Frequência	Como
1. Diário local	<code>pg_dump</code> comprimido	diário (madrugada)	cron no host → HD externo
2. Retenção	manter 14 dias	—	rotação de arquivos
3. Mídia externa	cópia para HD externo (e segundo HD alternado, um fora do prédio)	semanal	script + rotina do cliente
4. Offsite	cópia criptografada para nuvem	diário (quando há internet)	rclone (Backblaze B2/S3)

- **RPO**: até 24 h (pior caso). **RTO**: < 2–4 h (restaurar em qualquer máquina via Docker + dump).
- **Testar restore** a cada trimestre (restore real, não só "backup verde").
- Nobreak + NUT: queda de energia → shutdown gracioso → religa sozinho.

```
# exemplo de backup (cron diário)
docker exec estrela-db pg_dump -U estrela estrela_gestao | gzip > /backup/estrela_$(date + %F).sql.gz
```

6. Segurança

- Sistema **não exposto à internet**: portas fechadas no roteador; acesso externo só via **Tailscale** (ACL).
- Senhas argon2; JWT em cookie httpOnly; HTTPS interno (Caddy `tls internal`).
- Princípio do menor privilégio (RBAC, doc 03/05/06).
- Backup offsite **criptografado**.
- LGPD: dados de clientes minimizados; auditoria de operações; contrato operador (NextLayer) × controlador (Estrela).

7. Go-live (no cliente)

Sequência: 1. Vistoria elétrica/rede; instalar mini PC, nobreak, switch. 2. Subir stack (Docker Compose prod) + Caddy + Tailscale + monitoramento (Uptime Kuma ou ping + alerta). 3. **Migração definitiva** dos dados (ETL, doc 04) — rodar carga final após validar inconsistências com o cliente. 4. Configurar **modo aplicativo nos 10 terminais** (atalho `--app`, ícone Estrela Gestão). 5. Posicionar o **tablet do estoque** com a tela de localização em modo quiosque. 6. **Treinamento por perfil** (Admin, Vendedor, Financeiro, Funcionário). 7. **1 semana de operação assistida**; testar restore de backup no local.

Critério de aceite: 3 dias seguidos de operação real sem bloqueio + restore de backup validado.

8. Definition of Done do marco 08

- [] `docker-compose.prod.yml` + `Caddyfile` + `Dockerfile` funcionando (app + db + caddy).
- [] HTTPS interno em `https://sistema.local`; terminais acessam por nome.
- [] Backup diário automatizado + rotina offsite documentada; restore testado.
- [] Tailscale para manutenção; portas fechadas à internet.
- [] Procedimento de atualização e rollback documentado (`runbook-servidor.md`).

- [] Mini PC com religamento automático e nobreak com shutdown gracioso.
- [] 10 terminais em modo aplicativo + tablet de localização em quiosque.
- [] Operação assistida concluída e aceite assinado.

Dicionário de Dados — **CONTROLE.xlsx** (Estrela)

Mapa coluna→campo para o ETL (doc 04). Baseado na análise da planilha real entregue pelo cliente. Refinar com o cliente nas perguntas abertas (doc 01 §6).

Abas e seus papéis

Aba	Papel	Uso no ETL
CANETAS PLÁSTICAS	catálogo (categoria)	importar produtos
CANETAS METÁLICAS	catálogo	importar produtos
COPOS E GARRAFAS	catálogo	importar produtos
BLOCOS E CADERNOS	catálogo	importar produtos
CHAVEIROS	catálogo	importar produtos
ELETRÔNICOS	catálogo	importar produtos
OUTROS	catálogo	importar produtos
LUCIANO , CLAUDEMIR , LEONARDO	pedidos por cliente	histórico (opcional)
EM ABERTO	pendências de pagamento	contas a receber (futuro)
NOTAS	notas de fornecedor / entradas	custo + fornecedores
COLETA	dados da empresa	referência (não importar)
BOVESPA	anotações soltas	ignorar

Catálogo (abas de produto) — layout por blocos

Cabeçalho repetido a cada produto: CÓDIGO | DESCRIÇÃO | COD ALTERNATIVO | CORES | QUANTIDADE (+ colunas livres à direita).

Coluna planilha	Campo canônico	Regra de parsing
CÓDIGO (1ª linha do bloco)	produtos.codigo	string; identifica início do produto
CÓDIGO (número solto abaixo)	produtos.preco_pouca_qtd / preco_muita_qtd	1 número = 1 preço; 2 números = pouca/ muita qtd
DESCRIÇÃO (multi-linha)	produtos.descricao	concatenar linhas; extrair localização e "X em cada caixa"
COD ALTERNATIVO	produto_codigos_alt.codigo_alt	quando presente (ex.: K-820 → K-803)

Coluna planilha	Campo canônico	Regra de parsing
CORES (empilhada)	produto_variacoes.cor	uma variação por cor
QUANTIDADE	produto_variacoes.estoque_*	número→EXATO+saldo; "TEM MUITO/POUCO/TEM"→APROXIMADO+rótulo; "-"/"ACABOU"→ACABOU; "375 UNID"→EXATO+375
col. livre "2.000 EM CADA CAIXA"	produtos.unidades_por_caixa	extrair dígitos
col. livre "andar/lado/sala" (ou na descrição)	produtos.localizacao	extrair via regex de "andar"
col. livre "adicionado ao catálogo" / "chega em abril" / "PEDIDOS MUITO GRANDES"	produtos (observação)	texto livre
(aba de origem)	produtos.categoria_id	nome da aba → categoria

Rótulos de quantidade aproximada (normalização)

Texto na planilha	estoque_modo	rotulo_aprox
número (156 , 42)	EXATO	— (saldo = número)
375 UNID	EXATO	— (saldo = 375)
TEM MUITO	APROXIMADO	MUITO
TEM POUCO	APROXIMADO	POUCO
TEM / tem / TEM	APROXIMADO	TEM
– / ACABOU / vazio	APROXIMADO	ACABOU

Pedidos (abas de cliente) — opcional

Bloco: linha data | cliente | nº do pedido , depois CODIGO | DESCRICAO | QUANT. | V.UNIT | SUB.TOTAL até TOTAL .

Coluna	Campo	Observação
data / cliente / nº	pedidos.criado_em / cliente_id / numero	
CODIGO	resolver para produto (e cor pela descrição)	a cor está no nome (ex.: "STANLEY AZUL MARINHO")
QUANT.	pedido_itens.qtd	
V.UNIT	pedido_itens.preco_unit	mesmo SKU com preços diferentes por linha
SUB.TOTAL	pedido_itens.subtotal	conferência

Notas de fornecedor (NOTAS) — custo

Bloco com fornecedor + NF nº, depois linhas CODIGO | DESCRICAO | PREÇO | QUANTIDADE | RESTANTE
|

Coluna	Campo
fornecedor	fornecedores.nome
PREÇO	produtos.preco_custo (do item)
QUANTIDADE / RESTANTE	entrada de estoque / saldo de nota

Pontos de atenção para o parser

- Abas têm **número de colunas diferente** (7 a 13) — não assumir layout fixo; localizar colunas pelo header.
- Há **erros de digitação** nas notas livres ("adiicionado ao catálogo") — não depender de match exato.
- Alguns blocos têm **cor sem quantidade** ou **quantidade sem cor** → mandar para o relatório de inconsistências.
- Preço pode estar **ausente** em alguns produtos → marcar para revisão, não inventar.
- A planilha tem **~88 produtos** nas abas de catálogo; conferir contagem após import.

Runbook do Servidor — Estrela Gestão

Guia operacional do mini PC que roda o Estrela Gestão no cliente. Stack: PostgreSQL 16 + App (FastAPI/Gunicorn) + Caddy (HTTPS interno), tudo em Docker Compose. Acesso remoto de manutenção **somente via Tailscale**.

0. Convenções

- Diretório do projeto no servidor: `/opt/estrela` (ajuste se diferente).
- Arquivo de compose de produção: `docker-compose.prod.yml`.
- Variáveis sensíveis: `/opt/estrela/.env.prod` (NÃO versionado).
- Todos os comandos `docker compose` abaixo assumem:

```
cd /opt/estrela
export COMPOSE="docker compose -f docker-compose.prod.yml --env-file .env.prod"
```

1. Subir / derrubar a stack

```
# Subir tudo (db, app, caddy) em segundo plano
$COMPOSE up -d

# Conferir estado dos serviços
$COMPOSE ps

# Derrubar (mantém volumes/dados)
$COMPOSE down

# Derrubar e APAGAR volumes (PERIGO: apaga o banco) – só em recriação limpa
$COMPOSE down -v
```

O serviço `app` aplica as migrations (`alembic upgrade head`) automaticamente no entrypoint antes de iniciar o Gunicorn. Não é preciso rodar migrations à mão no fluxo normal.

2. Ver logs

```
# Logs de todos os serviços (segue em tempo real)
$COMPOSE logs -f

# Apenas o app
$COMPOSE logs -f app

# Apenas o banco
```



```
$COMPOSE logs -f db

# Últimas 200 linhas do app
$COMPOSE logs --tail=200 app
```

3. Restart e operações pontuais

```
# Reiniciar só o app (ex.: após ajuste de variável)
$COMPOSE restart app

# Reiniciar o Caddy (ex.: após editar Caddyfile)
$COMPOSE restart caddy

# Abrir um shell no container do app
$COMPOSE exec app bash

# Aplicar migrations manualmente (raro – só se desativar no entrypoint)
$COMPOSE exec app alembic upgrade head

# Abrir o psql no banco
$COMPOSE exec db psql -U estrela -d estrela_gestao
```

4. Atualização de versão (deploy)

Janela combinada com o cliente, fora do horário comercial. Acesso via Tailscale.

```
cd /opt/estrela

# 1) Trazer o novo código/imagem (git pull da tag desejada, ou pull de imagem)
git fetch --tags && git checkout vX.Y.Z

# 2) Reconstruir a imagem do app e subir
$COMPOSE build app
$COMPOSE up -d

# 3) As migrations rodam no entrypoint do app automaticamente.
# Conferir nos logs que "alembic upgrade head" terminou sem erro:
$COMPOSE logs --tail=100 app

# 4) Smoke test: abrir https://sistema.local e login; checar dashboard.
```

Faça um **backup manual antes de atualizar** (ver §6): `./scripts/backup-estrela.sh`.

5. Rollback para tag anterior

Se a nova versão quebrar:

```
cd /opt/estrela

# 1) Voltar o código para a tag estável anterior
```

```
git checkout vX.Y.(Z-1)

# 2) Reconstruir e subir
$COMPOSE build app
$COMPOSE up -d

# 3) Se a migration nova for incompatível e precisar reverter o schema:
$COMPOSE exec app alembic downgrade -1      # ou alembic downgrade <revisao>

# 4) Se o schema ficou inconsistente, restaurar o último dump bom (ver §7 do
#    disaster-recovery.md): para o app, restaura o dump, sobe o app.
```

Regra de ouro: migrations devem ser **retrocompatíveis** sempre que possível, para permitir rollback de código sem mexer no banco.

6. Backup

```
# Backup manual imediato (gera estrela_AAAAMMDD_HHMMSS.sql.gz em /backup/estrela_gestao)
/opt/estrela/scripts/backup-estrela.sh

# Backup automático: agendado no cron do host (madrugada)
# 0 2 * * * /opt/estrela/scripts/backup-estrela.sh >> /var/log/estrela-backup.log 2>&1

# Offsite criptografado (rclone) – só roda com internet
# 30 2 * * * /opt/estrela/scripts/backup-offsite.sh >> /var/log/estrela-offsite.log 2>&1
```

Teste de backup (trimestral)

1. Escolher um dump recente em `/backup/estrela_gestao`.
2. Restaurar em um banco/instância de teste (NUNCA em produção): - Subir um Postgres temporário ou usar máquina de homologação. - `DB_CONTAINER=<container-teste> ./scripts/restore-estrela.sh <dump>`
3. Validar: contagem de produtos/pedidos, login, dashboard.
4. Registrar a data do teste no controle interno.

7. Recuperação de falha do mini PC (RTO < 2 h)

Se o mini PC falhar (hardware):

1. Provisionar outra máquina com Docker + Docker Compose.
2. `git clone` do projeto em `/opt/estrela` (mesma tag em produção).
3. Recriar `/opt/estrela/.env.prod` (a partir do `.env.prod.example` + segredos do cofre).
4. Subir a stack: `$COMPOSE up -d` (cria banco vazio + migrations).
5. Restaurar o último dump bom: `bash $COMPOSE stop app ./scripts/restore-estrela.sh /backup/estrela_gestao/<ultimo>.sql.gz $COMPOSE start app`
6. Reapontar o DNS/IP fixo de `sistema.local` para a nova máquina.
7. Smoke test nos terminais.

Detalhes completos em `disaster-recovery.md`.

8. Saúde e diagnóstico rápido

```
# Containers de pé?
$COMPOSE ps

# Banco aceitando conexão?
$COMPOSE exec db pg_isready -U estrela -d estrela_gestao

# Caddy servindo?
curl -k https://localhost/ -I

# Uso de disco (atenção ao /backup e ao volume do Postgres)
df -h
docker system df
```

9. Contatos

Papel	Responsável	Contato
Suporte / Operadora	NextLayer (DevOps)	(preencher: telefone / e-mail / canal)
Responsável Estrela	(preencher)	(preencher)
Tailscale / acesso remoto	NextLayer	tailnet: (preencher)
Provedor de internet	(preencher)	(preencher)

Manter este quadro atualizado. Em incidente: 1) abrir acesso Tailscale, 2) coletar logs (§2), 3) acionar o contato de suporte.

Checklist de Go-Live — Estrela Gestão

Passo a passo para colocar o sistema em produção no cliente (mini PC + 10 terminais). Marque cada item. Não avance de bloco sem concluir o anterior.

Fase 1 — Pré-requisitos (antes de ir ao cliente)

- ☐ Tag de versão estável definida e testada em homologação.
- ☐ Imagem do app builda sem erro (`docker compose -f docker-compose.prod.yml build`).
- ☐ Planilhas reais do cliente coletadas e validadas no ETL (relatório de inconsistências aceito).
- ☐ Segredos gerados (`DB_PASSWORD` , `JWT_SECRET`) e guardados no cofre.
- ☐ Tailscale com conta/ACL prontos para o mini PC.
- ☐ HD externo para backup formatado e identificado.

Fase 2 — Vistoria física (no cliente)

- ☐ **Vistoria elétrica:** tomada aterrada, circuito estável para o mini PC + nobreak.
- ☐ **Nobreak senoidal 1500 VA** instalado; mini PC e switch ligados nele.
- ☐ Cabo USB do nobreak conectado ao mini PC (para shutdown via NUT).
- ☐ **Rede:** switch gigabit posicionado; terminais cabeados ou Wi-Fi estável.
- ☐ Mini PC com **IP fixo** na rede interna.
- ☐ BIOS configurada: **"Power On After Power Loss" = On** (religa após queda).

Fase 3 — Subir a stack

- ☐ Instalar Docker + Docker Compose no mini PC.
- ☐ `git clone` do projeto em `/opt/estrela` (tag de produção).
- ☐ Criar `/opt/estrela/.env.prod` a partir de `.env.prod.example` (preencher segredos).
- ☐ Subir: `docker compose -f docker-compose.prod.yml --env-file .env.prod up -d`.
- ☐ Conferir migrations no log do app: `... logs --tail=100 app` (alembic upgrade head OK).
- ☐ DNS/mDNS: `sistema.local` resolve para o IP fixo do mini PC (roteador ou Avahi).
- ☐ HTTPS interno: `https://sistema.local` abre com certificado interno do Caddy.
- ☐ Instalar/confiar na CA interna do Caddy nas estações (evita aviso de certificado).

Fase 4 — Migração definitiva dos dados (ETL)

- ☐ Backup do estado vazio (referência).
- ☐ Rodar a carga final (idempotente) com as planilhas validadas: `bash docker compose -f docker-compose.prod.yml exec app \ uv run python scripts/import_planilhas.py --file /caminho/planilha.xlsx` (rodar antes `--dry-run` para conferir o relatório de inconsistências)
- ☐ Conferir totais: nº de produtos, saldos de estoque iniciais (movimentação tipo importação), clientes.

- [] Backup imediato pós-carga: `./scripts/backup-estrela.sh`.
-

Fase 5 — Terminais em modo aplicativo (10 estações)

Para cada terminal:

- [] Navegador instalado (Chrome ou Edge).
- [] Criar atalho em **modo aplicativo** apontando para o sistema:
`chrome --app=https://sistema.local` (Edge: `msedge --app=https://sistema.local`)
- [] Ícone do atalho na área de trabalho com a marca **Estrela Gestão**.
- [] Testar: abre em tela cheia, sem barra de navegação ("cara de programa"), login funciona.
- [] PWA instalável: confirmar manifest/ícone e service worker (network-first) ativos.

Tablet de localização (quiosque)

- [] Tablet posicionado no estoque, ligado à rede interna.
 - [] Abrir a tela `/estoque/localizacao` em **modo quiosque** (tela cheia, sem sair do app).
 - [] Bloquear navegação para fora; manter ligado/carregando.
-

Fase 6 — Backup e acesso remoto

- [] Cron de backup diário ativo (`backup-estrela.sh`, madrugada) → HD externo.
 - [] Cron de offsite (`backup-offsite.sh`) configurado (`rclone crypt`).
 - [] **Teste de restore real** validado em máquina/instância de teste.
 - [] Tailscale conectado no mini PC; ACL restrita; portas do roteador **fechadas** à internet.
 - [] Monitoramento (Uptime Kuma ou ping + alerta) apontando para o mini PC.
-

Fase 7 — Treinamento por perfil

- [] **Admin**: cadastros, importação, usuários, auditoria, relatórios completos.
 - [] **Vendedor**: novo pedido, busca de estoque, clientes (sem ver preço de custo).
 - [] **Financeiro**: faturar, contas a receber, baixas, relatórios financeiros.
 - [] **Funcionário**: entradas de mercadoria, fila de separação, inventário (sem ver custo).
 - [] Entregar guia rápido por perfil (1 página cada).
-

Fase 8 — Operação assistida e aceite

- [] **1 semana de operação assistida** (presencial/remoto via Tailscale).
 - [] Acompanhar volume real (80–100 pedidos/dia) sem bloqueios.
 - [] **Critério de aceite**: 3 dias seguidos de operação real sem bloqueio + restore de backup validado no local.
 - [] Termo de aceite assinado pelo cliente.
-

Pós go-live

- [] Agendar teste de restore trimestral.
- [] Revisar logs/alertas na primeira semana e ao fim do mês.
- [] Atualizar contatos no `runbook-servidor.md`.

Plano de Recuperação de Desastres (DR) — Estrela Gestão

Como recuperar o sistema após uma falha (hardware, corrupção de dados, perda do local).
Cenário: sistema 100% local em mini PC, sem RAID/ECC. A estratégia de DR é **backup + restore rápido** em qualquer máquina com Docker.

1. Objetivos (RPO / RTO)

Métrica	Alvo	Como é garantido
RPO (perda máxima de dados)	até 24 h (pior caso)	Backup <code>pg_dump</code> diário (madrugada) + offsite criptografado diário quando há internet
RTO (tempo de retorno)	< 2 h (falha de mini PC) / < 4 h (perda total do local)	Restore de dump via Docker em máquina nova

Reduzir o RPO abaixo de 24 h exige backups mais frequentes (ex.: a cada 4 h). Avaliar com o cliente se o volume justificar. Hoje, a janela de 24 h é a aceita no escopo enxuto.

2. Camadas de backup (3-2-1 adaptado)

1. **Diário local:** `pg_dump` | `gzip` em `/backup/estrela_gestao/estrela_AAAAMMDD_HHMMSS.sql.gz` (HD externo). Retenção 14 dias.
2. **Mídia externa rotacionada:** cópia semanal para segundo HD, um mantido **fora do prédio**.
3. **Offsite na nuvem:** `rclone sync` criptografado (remote `b2_encrypted` — Backblaze B2/S3), diário quando há internet.

Scripts: `scripts/backup-estrela.sh`, `scripts/backup-offsite.sh`, `scripts/restore-estrela.sh`.

3. Cenários e procedimentos

3.1 Falha de hardware do mini PC (RTO < 2 h)

1. Provisionar máquina substituta com **Docker + Docker Compose**.
2. `git clone` do projeto em `/opt/estrela` na **mesma tag** que estava em produção.
3. Recriar `/opt/estrela/.env.prod` (do `.env.prod.example` + segredos do cofre).
4. Subir a stack: `bash cd /opt/estrela docker compose -f docker-compose.prod.yml --env-file .env.prod up -d` Isso cria o banco vazio e aplica migrations (entrypoint).
5. Restaurar o último dump válido: `bash docker compose -f docker-compose.prod.yml stop app ./scripts/restore-estrela.sh /backup/estrela_gestao/<ultimo>.sql.gz docker compose -f docker-compose.prod.yml start app`
6. Reapontar `sistema.local` (IP fixo) para a nova máquina (roteador/Avahi).
7. Validar (ver §4).

3.2 Corrupção / perda lógica de dados (exclusão indevida, etc.)

1. Identificar o último dump **anterior ao incidente**.
2. `docker compose ... stop app` (evita escrita concorrente).
3. Restaurar esse dump (`restore-estrela.sh`).
4. `start app` e validar.

O dump restaura o banco inteiro. Restauração parcial (uma tabela) exige extração manual do dump.

3.3 Perda total do local (incêndio, furto) — RTO < 4 h

1. Obter o backup **offsite** (rclone) numa máquina nova com internet: `bash rclone sync b2_encrypted:estrela_gestao /backup/estrela_gestao`
2. Seguir o procedimento de **3.1** (máquina nova) usando o dump baixado.
3. Reconfigurar rede/terminais conforme `go-live-checklist.md` (fases 2, 3 e 5).

4. Validação pós-restore (obrigatória)

Após qualquer restore, confirmar:

- [] App sobe e `https://sistema.local` responde.
- [] Login funciona (um usuário de cada perfil).
- [] Dashboard carrega KPIs.
- [] Contagem de **produtos**, **pedidos** e **clientes** bate com o esperado: `bash docker compose -f docker-compose.prod.yml exec db \ psql -U estrela -d estrela_gestao -c \ "select 'produtos' t, count(*) from produtos union all select 'pedidos', count(*) from pedidos union all select 'clientes', count(*) from clientes;"`
- [] Saldos de estoque coerentes (sem negativos inesperados).
- [] Registrar data/horário do restore e responsável.

5. Teste de DR (trimestral)

- Fazer um restore **real** (não apenas "backup verde") em instância de teste.
- Cronometrar o tempo total (validar o RTO).
- Documentar problemas e ajustar este plano.

6. Itens críticos a proteger fora do servidor

- `.env.prod` (segredos `DB_PASSWORD`, `JWT_SECRET`) — guardar em cofre de senhas.
- Configuração do remote rclone (`rclone.conf`) com a chave de criptografia do crypt.
- Esta documentação e o `runbook-servidor.md`.

Sem a chave do rclone crypt, o backup offsite é **irrecuperável** (por design). Guardar com cuidado.

LGPD — Responsabilidades do Operador — Estrela Gestão

Documento de referência sobre o tratamento de dados pessoais no Estrela Gestão, conforme a **Lei nº 13.709/2018 (LGPD)**. Define papéis, princípios aplicados e medidas técnicas. **Não substitui parecer jurídico** — é a base operacional acordada entre as partes.

1. Papéis (art. 5º, VI e VII)

Papel	Quem	Responsabilidade
Controlador	Estrela (empresa cliente)	Decide sobre as finalidades e os meios do tratamento. É a titular da relação com os dados de seus clientes/fornecedores.
Operador	NextLayer	Realiza o tratamento em nome do controlador , conforme instruções. Fornece e mantém o sistema, infraestrutura e backups.

A NextLayer (operadora) **não usa os dados para finalidade própria**. Trata apenas o necessário para operar, manter e dar suporte ao sistema, sob instrução da Estrela (controladora).

2. Dados pessoais tratados (minimização — art. 6º, III)

O sistema é de **estoque e pedidos** (B2B atacado). Dados pessoais são limitados ao necessário:

- **Clientes:** nome/razão social, documento (CNPJ/CPF), contato, condição de pagamento, endereço de entrega.
- **Usuários internos:** nome, login, perfil de acesso, senha (hash argon2).
- **Operações:** quem fez o quê (auditoria) — vinculado ao usuário interno.

Não são tratados dados sensíveis (art. 5º, II). Caso o cliente queira incluir, deve ser reavaliado.

Princípio aplicado: coletar e reter **apenas o mínimo** para a finalidade de gestão comercial.

3. Medidas de segurança (art. 46)

- **Sem exposição à internet:** sistema roda em rede interna; acesso remoto de manutenção só via **Tailscale** (rede privada, ACL).
- **Controle de acesso (RBAC):** 4 perfis (Admin, Vendedor, Financeiro, Funcionário); menor privilégio. Preço de custo oculto para Vendedor/Funcionário.
- **Autenticação:** senhas com **argon2**; sessão via **JWT em cookie httpOnly**.
- **HTTPS interno:** Caddy com `tls internal` (criptografia no tráfego da rede local).
- **Backup criptografado offsite:** rclone com remote **crypt**; chave guardada em cofre (sem ela o backup é irrecuperável).
- **Backup local** em HD externo, com rotação; restauração testada periodicamente.

4. Auditoria e rastreabilidade (art. 37 — registro das operações)

- Tabela de **auditoria** registra antes/depois (JSONB) em produtos, pedidos, estoque e financeiro, com **usuário, data/hora e origem**.
 - **Estoque é append-only**: toda alteração de saldo gera movimentação imutável (não há `UPDATE` direto).
 - Logs do servidor e do app disponíveis para investigação de incidentes.
-

5. Direitos dos titulares (art. 18)

A Estrela (controladora) é o ponto de contato dos titulares. A NextLayer (operadora) **apoia tecnicamente**:

- **Acesso / correção**: dados de clientes editáveis na tela de Clientes (perfil autorizado).
 - **Eliminação / anonimização**: mediante solicitação formal da controladora; considerar obrigações legais de retenção fiscal antes de excluir (pedidos/financeiro).
 - **Portabilidade**: exportação de dados (XLSX/relatórios) disponível no sistema.
-

6. Incidentes de segurança (art. 48)

- Em caso de incidente com risco a titulares, a **NextLayer comunica a Estrela imediatamente**.
 - A Estrela (controladora) avalia a comunicação à **ANPD** e aos titulares.
 - Coletar evidências (logs §4 do runbook), conter, restaurar de backup íntegro se necessário.
-

7. Retenção e término

- Dados retidos enquanto durar a relação comercial e as obrigações legais (fiscais/contábeis).
 - Ao encerrar o contrato, a NextLayer **devolve ou elimina** os dados conforme instrução da Estrela, salvo guarda exigida por lei (art. 16).
 - Backups offsite seguem a mesma política; a chave de criptografia é destruída no encerramento.
-

8. Resumo das obrigações por parte

Tema	Estrela (Controladora)	NextLayer (Operadora)
Finalidade do tratamento	Define	Segue instruções
Atendimento ao titular	Responsável	Apoio técnico
Comunicação à ANPD	Responsável	Notifica a Estrela
Segurança técnica	Acompanha	Implementa e mantém
Backup/criptografia	Acompanha	Executa
Acesso de funcionários internos	Define perfis	Implementa RBAC

Nobreak + NUT — Shutdown Gracioso — Estrela Gestão

Configuração do **NUT (Network UPS Tools)** no mini PC para que, em queda de energia, o servidor desligue de forma **graciosa** antes da bateria do nobreak acabar — e religue sozinho quando a energia voltar. Protege a integridade do PostgreSQL.

1. Hardware esperado

- **Nobreak senoidal 1500 VA** (ex.: NHS / Intelbras) com porta **USB** de comunicação.
- Cabo USB do nobreak ligado ao mini PC.
- Mini PC, switch e roteador alimentados pelo nobreak.

2. Pré-requisitos no BIOS (crítico)

- **"Power On After Power Loss" = On** (ou "Restore on AC Power Loss" = Power On). Assim, quando a energia retorna, o mini PC **liga sozinho** e o Docker (`restart: always`) sobe a stack.
- Desabilitar suspensão automática / hibernação no SO.

3. Rede do servidor

- Mini PC com **IP fixo** na rede interna (ex.: `192.168.0.10`). Necessário para:
- `sistema.local` resolver sempre para o mesmo IP.
- O `upsmon` local falar com o `upsd` em `localhost` .

4. Instalar o NUT

```
sudo apt-get update
sudo apt-get install -y nut
```

Modo de operação: **standalone** (um único servidor monitora seu próprio nobreak).

5. Arquivos de configuração

Caminhos típicos em Debian/Ubuntu: `/etc/nut/` . Ajuste o `driver / port` ao seu modelo (use `nut-scanner -U` para detectar o nobreak USB).

5.1 /etc/nut/nut.conf

```
MODE=standalone
```

5.2 /etc/nut/ups.conf

```
[estrela]
    driver = usbhid-ups
    port = auto
    desc = "Nobreak Estrela 1500VA"
    # Se o modelo não for reconhecido, testar driver/vendorid específicos:
    # vendorid = ....
    # productid = ....
```

5.3 /etc/nut/upsd.conf

```
# Apenas localhost – não expor o NUT na rede.
LISTEN 127.0.0.1 3493
```

5.4 /etc/nut/upsd.users

```
[monuser]
    password = TROQUE_ESTA_SENHA
    upsmon master
```

5.5 /etc/nut/upsmon.conf

```
# Monitora o nobreak local. "master" = este host desliga o nobreak no fim.
MONITOR estrela@localhost 1 monuser TROQUE_ESTA_SENHA master

# Mínimo de fontes de energia necessárias.
MINSUPPLIES 1

# Comando de desligamento do sistema (shutdown gracioso).
SHUTDOWNCMD "/sbin/shutdown -h +0"

# Tempo (s) que aguarda após bateria crítica antes de forçar o desligamento.
FINALDELAY 5

# Notificações (opcional): chamar upssched para avisos.
NOTIFYCMD /usr/sbin/upssched
```

Política de shutdown: o NUT dispara o desligamento quando o nobreak sinaliza **bateria baixa** (evento LB — Low Battery), garantindo que o SO desligue antes da bateria zerar. Não esperar a bateria acabar: configurar o nobreak (ou o threshold) para um nível de segurança.

6. Habilitar e iniciar

```
sudo systemctl enable --now nut-server
sudo systemctl enable --now nut-monitor

# Conferir comunicação com o nobreak
upsc estrela@localhost
# Esperado: linhas como battery.charge, ups.status (OL = on line / OB = on battery)
```

7. Sequência em queda de energia

```
Energia cai
|
▼
Nobreak entra em bateria (ups.status = OB) → sistema segue rodando
|
▼ (bateria atinge nível baixo: ups.status = OB LB)
upsmon dispara SHUTDOWNCMD
|
▼
SO desliga graciosamente (Docker para os containers; Postgres encerra limpo)
|
▼
Energia volta → BIOS "Power On After Power Loss" liga o mini PC
|
▼
Docker (restart: always) sobe db + app + caddy; app roda migrations e atende
```

8. Teste de validação

1. Com o sistema rodando, **tirar o nobreak da tomada** (energia da rua).
2. Confirmar `ups.status = OB` em `upsc estrela@localhost`.
3. Aguardar até bateria baixa (ou simular) e observar o **shutdown automático**.
4. Religar a energia e confirmar que o mini PC **liga sozinho** e a stack sobe.
5. Validar acesso a `https://sistema.local` e integridade do banco.

Fazer este teste no go-live e revisar anualmente (baterias degradam).

9. Observações

- Trocar as senhas placeholder (`TROQUE_ESTA_SENHA`) por valores reais e guardá-las no cofre.
- Não expor o NUT na rede (`LISTEN 127.0.0.1`); o monitoramento é local.
- Registrar o modelo do nobreak e o driver usado neste documento após a instalação.